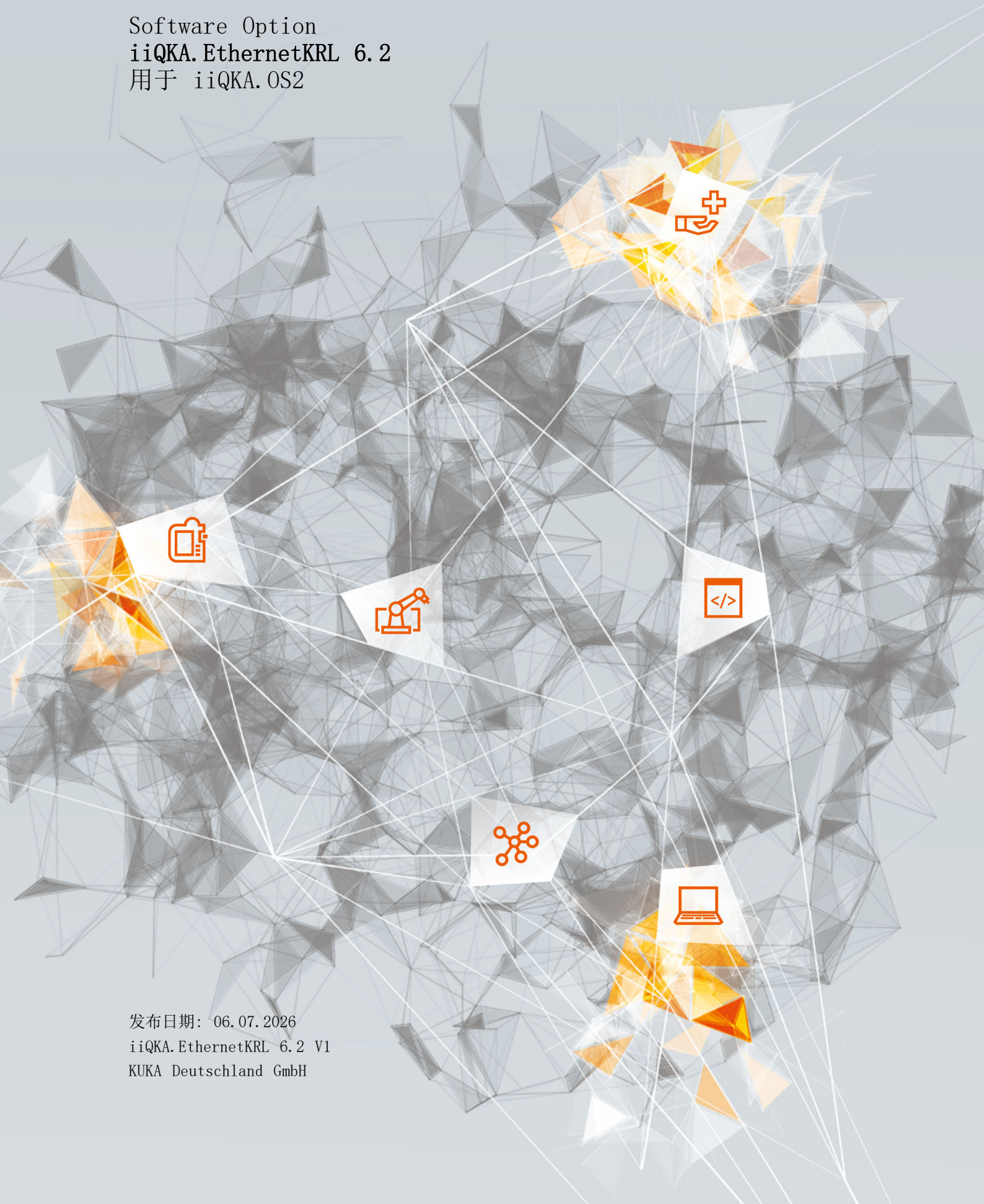
Software Option

# iiQKA.EthernetKRL 6.2

用于 iiQKA.OS2



发布日期: 06.07.2026

iiQKA.EthernetKRL 6.2 V1

KUKA Deutschland GmbH

©版权说明 2026
KUKA Deutschland GmbH
Zugspitzstraße 140
D-86165 Augsburg
德国

此文献或节选只有在征得 KUKA Deutschland GmbH 明确同意的情况下才允许复制或对第三方开放。
除了本文献中说明的功能外，控制系统还可能具有其他功能。 但是在新供货或进行维修时，无权要求 KUKA Deutschland GmbH 提供这些功能。
我们已就印刷品的内容与描述的硬件和软件内容是否一致进行了校对。 但是不排除有不一致的情况，我们对此不承担责任。 但是我们定期校对印刷品的内容，并在之后的版本中作必要的更改。
我们保留在不影响功能的情况下进行技术更改的权利。
KIM-PS5-DOC
原版文件的译文

| | |
|---|---|
| 出版物: | Pub iiQKA.EthernetKRL 6.2 (PDF) zh |
| | PB27901 |
| 书页构造: | iiQKA.EthernetKRL 6.2 V1.1 |
| | BS24390 |
| 版本: | iiQKA.EthernetKRL 6.2 V1 |

# 目录


1 引言 7
1.1 目标群体 7
1.2 工业机器人文档 7
1.3 安全提示和注意事项的图示说明 7
1.4 所用术语 8
1.5 许可证条件 9
2 产品说明 11
2.1 产品说明 11
2.2 工作原理 11
2.3 传输协议 12
2.4 客户端-服务器运行模式 12
2.5 事件信息 13
2.6 故障情况下的行为 13
2.7 错误处理 13
2.8 预期用途和违规使用 13
3 安全 15
4 网络安全 17
4.1 网络安全提示 17
5 安装 19
5.1 系统要求 19
5.2 安装或更新 iiQKA.EthernetKRL 19
5.3 iiQKA.EthernetKRL 卸载 20
6 操作 21
6.1 管理以太网配置 21
6.2 更改以太网配置的 IP 地址或端口 22
7 投入运行和配置 23
7.1 概览：投入运行和配置 23
7.2 通过 KLI 建立网络连接 23
7.3 创建和编辑以太网配置 23
7.3.1 一般设定 25
7.3.2 接收配置 27
7.3.2.1 测试接收配置 31
7.3.3 发送配置 31
7.3.3.1 测试发送配置 32
7.4 将以太网配置传输到机器人控制器 32
8 编程 33
8.1 编程提示 33
8.2 KRL 函数 33
8.2.1 EKI_Load：加载以太网配置 35
8.2.2 EKI_Open：打开通信通道 36
8.2.3 EKI_Close：关闭通信通道 37

8.2.4 EKI_Unload：卸载以太网配置 37
8.2.5 EKI_DeleteReceivedData：删除接收到的数据 38
8.2.6 EKI_ReadNext：选择下一个数据包 38
8.2.7 EKI_GetBool：读取布尔值 39
8.2.8 EKI_GetBoolArray：读取 BOOL 数组 39
8.2.9 EKI_GetInt：读取整数值 40
8.2.10 EKI_GetIntArray：读取 INT 数组 41
8.2.11 EKI_GetReal：读取实数值 42
8.2.12 EKI_GetRealArray：读取 REAL 数组 43
8.2.13 EKI_GetString：读取字符串 44
8.2.14 EKI_GetStringArray：读取字符串数组 44
8.2.15 EKI_GetFrame: 读取 KRL 坐标系 45
8.2.16 EKI_GetFrameArray：读取 KRL 坐标系数组 46
8.2.17 EKI_Send：发送数据 47
8.2.18 EKI_SetBool：设置布尔值 48
8.2.19 EKI_SetInt：设置整数值 49
8.2.20 EKI_SetReal：设置实数值 49
8.2.21 EKI_SetString：设置字符串 50
8.2.22 EKI_SetFrame：设置 KRL 坐标系 51
8.2.23 EKI_GetBufferedMsgCount：读取已接收消息的数量 51
8.2.24 EKI_GetConnectionState：查询连接状态 52
8.2.25 EKI_IsClosed：检查通道是否已关闭 53
8.2.26 EKI_IsWaiting：检查通道是否处于等待状态 53
8.2.27 EKI_IsActive：检查通道是否处于激活状态 53
8.2.28 EKI_IsLoaded：检查通道是否已加载 54
8.2.29 EKI_IsInState：检查通道是否处于预期状态 54
8.2.30 EKI_HasValue：检查可选值是否存在 55
8.2.31 EKI_HasFrameValue：检查是否存在完整的 KRL 坐标系 56
8.2.32 更改信息级别 56
8.3 打开和关闭以太网连接 57
8.4 读取收到的数据 58
8.5 读取数组数据 60
8.6 发送数据 61
**9 仿真 63**
9.1 使用 RCS 进行仿真 63
**10 示例 67**
10.1 示例概述 67
10.1.1 准备服务器程序和示例 68
10.1.2 服务器程序中的操作和设置 69
10.1.3 “XML”示例 71
10.1.4 “Binary”示例 73
10.1.5 “String”示例 74
**11 诊断 77**
11.1 使用诊断监视器进行通道诊断 77
**12 附录 79**
12.1 迁移至 iiQKA.EthernetKRL 6.x 的编程说明 79

12.1.1 通用变更 79
12.1.2 KRL 函数变更 79
13 KUKA 客户服务部 81
索引 83

# 1 引言

## 1.1 目标群体

本文档针对具有下列知识的用户：

- KRL 编程的高级知识
- 机器人控制器方面的高级系统知识
- XPath 和正则表达式的基础知识
- 高级网络知识

为确保最佳使用 KUKA 产品，我们建议您参加 KUKA College 的培训课程。有关培训计划的信息请访问公司主页 www.kuka.com 或直接在公司分支机构处获取。

## 1.2 工业机器人文档

工业机器人文档由以下几部分组成：

- 机器人本体文档
- 机器人控制器文档
- KUKA smartPLUG 或 KUKA smartPAD 文档（如果使用）
- 系统软件文档
- 选件及附件说明
- KUKA Xpert 中的备件概览

每份说明均为独立文件。

## 1.3 安全提示和注意事项的图示说明

### 安全

这些安全提示事关安全，**必须**遵守。

**危险**

这类安全提示表示：如果未采取预防措施，则肯定或极有可能**会**导致死亡或严重伤害。

**警告**

这类安全提示表示：如果未采取预防措施，**可能会**导致死亡或严重伤害。

**小心**

这类安全提示表示：如果未采取预防措施，**可能会**导致轻伤。

**注意**

这类安全提示表示：如果未采取预防措施，**可能会**导致财产损失。

这类安全提示提及安全相关信息或一般安全措施。
这类安全提示并不涉及具体危险或具体安全措施。

本安全提示提醒注意用于预防或补救紧急情况或故障的操作步骤：

**安全须知**

必须严格遵守以下操作步骤！

**必须**严格遵守带有此安全提示标识的操作步骤。

**提示**

这类提示可使您工作更加便利或提供进一步信息的说明。



有助于您轻松工作或者包含进一步信息的提示。

## 1.4 所用术语

| 术语 | 说明 |
|---|---|
| EKI | EthernetKRL Interface（以太网 KRL 接口） |
| 以太网 | 局部数据网络的数据网络技术<br>以太网以数据框架的形式实现连接用户之间的数据交换。 |
| FIFO | First In First Out（先进先出）<br>数据存储和读取的顺序：在 FIFO 中，最先存储的元素最先被取出。 |
| IP | Internet Protocol（互联网协议）<br>互联网协议的任务是通过物理 MAC 地址定义子网络。 |
| iiQWorks.App Builder | 用于创建备选软件包的软件包 |
| iiQWorks.Sim | 受控于 KR C 的机器人单元的仿真和工程环境 |
| KLI | KUKA Line Interface（KUKA 线路接口）<br>机器人控制器用于外部通讯的以太网接口（无实时功能） |
| KRL | KUKA Robot Language（库卡机器人编程语言）<br>KUKA 机器人编程语言 |
| LIFO | Last In First Out（后进先出）<br>数据存储和读取的顺序：在 LIFO 中，最后存储的元素最先被取出。 |
| 套接字 | 将 IP 地址和端口号相互连接的软件接口 |
| TCP | Transmission Control Protocol（传输控制协议）<br>有关网络用户间数据交换的协议。TCP 可在网络连接的 2 个终点之间建立一个虚拟通道。在这个通道上可双向传输数据。 |
| UDP | 用户数据报协议<br>用于网络设备间数据交换的无连接协议。<br>UDP 未与网络建立连接。 |
| XML | Extensible Markup Language（可扩展的标记语言）<br>一种基于文本的标记格式，用于数据的结构化表示和交换。 |
| XPath | XML Path 语言<br>一种查询语言，用于在 XML 和 HTML 文档中导航和选择元素、属性及文本内容。 |

## 1.5 许可证条件

KUKA 许可证条件和所用开源软件的许可证条件可在以下位置找到：

- 在 iiQKA.UI 操作界面上：主菜单 > **系统** > **系统信息** > **许可证**
- 在 iiQWorks.Sim 中：**文件** > **选项** > **插件** > **管理** > **显示许可证**
- 在 iiQWorks.Cockpit 中：**软件存储库** > **详细信息** > **显示许可证**



关于开源许可证的更多信息可通过下面的邮件地址获取：
opensource@kuka.com

引言

# 2 产品说明

## 2.1 产品说明

### 说明

iiQKA.EthernetKRL 是用于在提交解释器或机器人解释器程序运行时通过以太网在机器人控制器和外部系统之间进行数据交换的备选软件包。

### 功能

- 接收外部系统的 XML 数据
- 将 XML 数据发送给外部系统
- 接收外部系统的非结构化文本
- 向外部系统发送非结构化文本

## 2.2 工作原理

### 说明

通过 iiQKA.EthernetKRL 备选软件包，可以在机器人控制器与外部网络设备之间进行数据交换。在此过程中，既可以将数据发送给外部设备，也可以从该设备接收数据。

以太网配置定义了通信的工作方式。其中包括（但不限于）以下设置：

- 连接类型及所使用的传输协议
- 待传输数据必须采用的格式
- 从传入的消息中提取和解释数据的规则

(>>> *7.3 "创建和编辑以太网配置" 页面 23*)

无论选择何种协议或连接类型，数据发送与接收的基本流程始终保持不变。

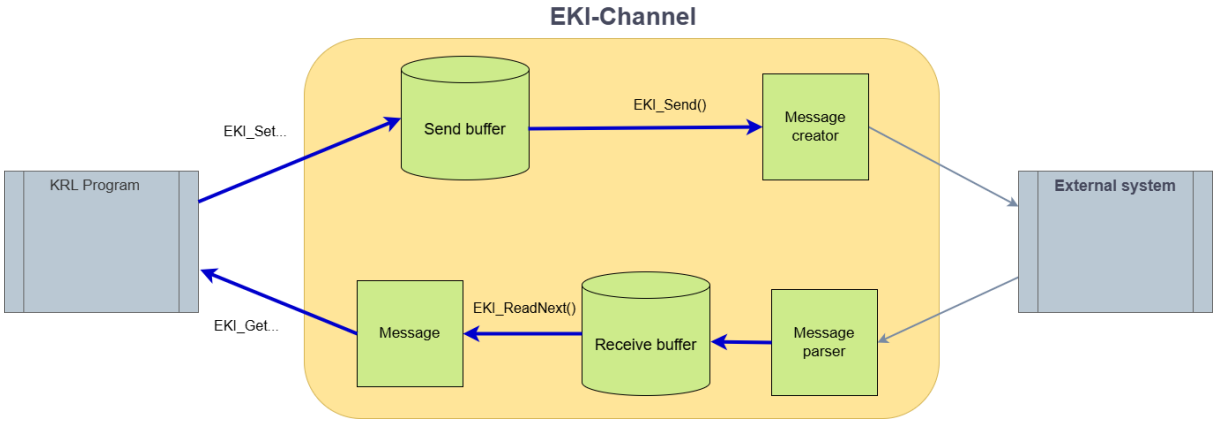



图 2-1: 工作原理

### 数据接收

一旦通过 EKI 接口接收到数据，系统便会尝试提取出一个完整的消息包。为此，使用在以太网配置中定义的正则表达式 (Regex) 作为分隔符。

当识别到一条完整消息时，将从消息中提取数值。该提取过程使用已定义消息变体中的规则。以此方式获取的数值随后将作为一个封闭单元存入接收缓冲区中。

- 通过 EKI_ReadNext 函数，系统会为 KRL 提供待读取的下一个值，同时将其从接收缓冲区中删除。
- 随后，可使用 EKI_Get 函数在 KRL 中访问已接收消息的所有数值。

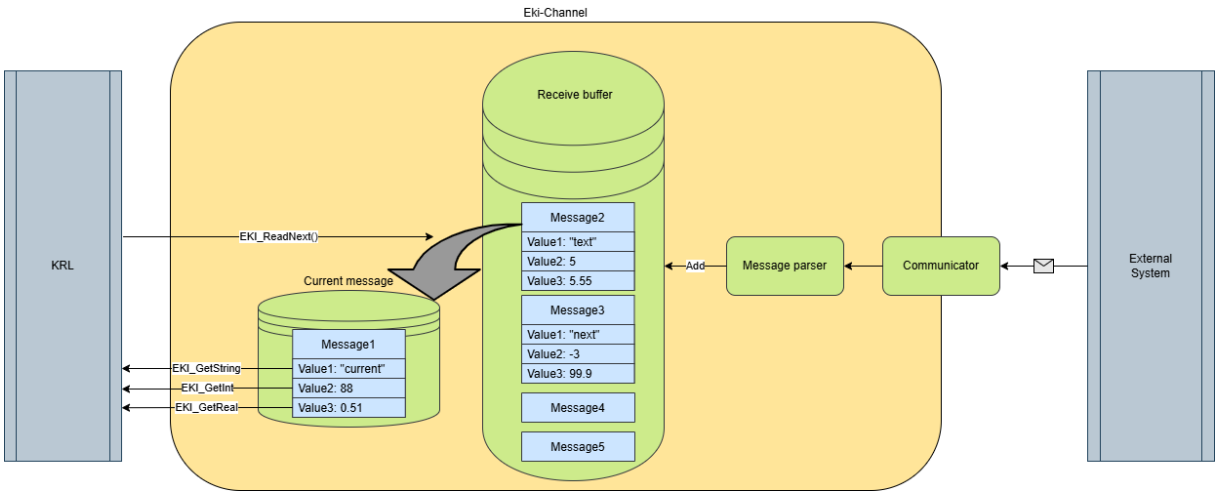



图 2-2: 数据接收

### 发送数据

通过 EKI_Set 函数可将对端数据写入发送缓冲区。当所有数据传输至发送缓冲区后，EKI_Send 函数会将其转换为定义的消息并发送至对端。

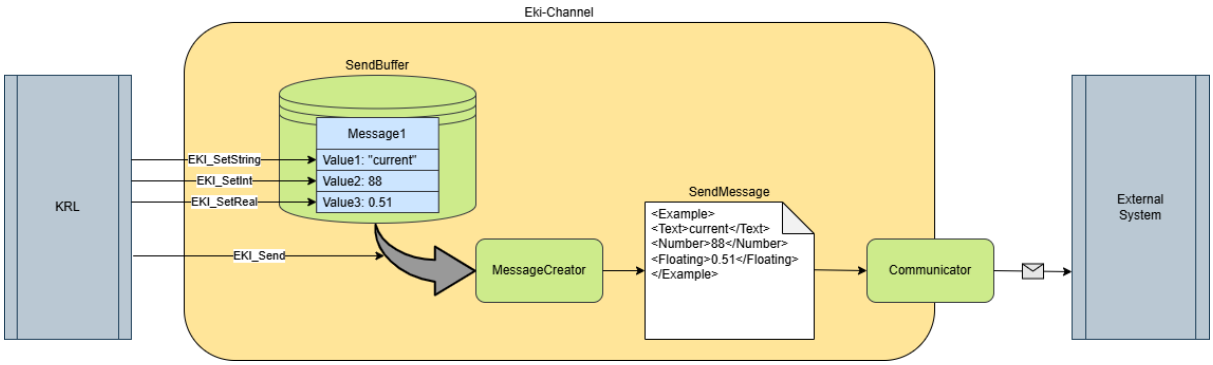



图 2-3: 发送数据

## 2.3 传输协议

数据在默认情况下通过 TCP/IP 协议传输。可以使用 UDP/IP 协议，但不建议这样做（无连接网络协议，例如无法识别数据丢失）。

## 2.4 客户端-服务器运行模式

机器人控制系统和外部系统作为客户端和服务器相互连接。其中，外部系统可以是客户端或服务器。激活的连接数量限制在 16 个。

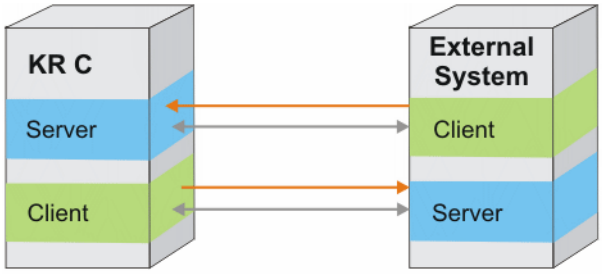



图 2-4: 客户端-服务器运行模式

将 EKI 配置为服务器时，只能有一个单独的客户端与服务器连接。如果需要多个连接，则还要创建多台 EKI 服务器。可以同时在 EKI 内部运行多个客户端和服务器。

## 2.5 事件信息

在以太网配置中，可以选择为以下事件定义设置输出或标志：

- 通信错误
  (>>> *7.3.1 "一般设定" 页面 25*)
- 接收到完整消息
  (>>> *7.3.2 "接收配置" 页面 27*)

## 2.6 故障情况下的行为

EKI 接口的以下特性确保了即使在发生故障时，接收到的数据也能得到可靠处理：

- 达到存储上限时：当接收缓冲区达到存储上限时，可以应用预先配置的策略。
  (>>> *"专家设置" 页面 26*)
- 无效消息：如果无法从传入的数据中提取出有效的消息，连接将被自动关闭。
  (>>> *7.3.2 "接收配置" 页面 27*)
- 连接关闭后继续读取：即使在连接关闭之后，已存储的数据仍可以继续被读取。
- 连接中断后的恢复：中断的连接可以在不影响已存储数据的情况下恢复。
- 连接中断的通知：连接中断可以通过例如标志进行报告。
  (>>> *"专家设置" 页面 26*)

## 2.7 错误处理

备选软件包提供了多种 KRL 函数。这些函数中的大多数均返回一个错误代码作为返回值。可在 KRL 程序中查询返回值，以检查函数是否正确执行。
(>>> *8.2 "KRL 函数" 页面 33*)
执行 KRL 函数时可以输出信息。可配置需要显示的消息类型。
(>>> *8.2.32 "更改信息级别" 页面 56*)

## 2.8 预期用途和违规使用

### 应用

备选软件包 iiQKA.EthernetKRL 是为在提交解释器或机器人解释器程序运行时通过以太网在机器人控制器和外部系统之间进行数据交换而设计的。
如果该备选软件包用于通过外部操作设备控制机器人（例如操纵杆、带用户界面的笔记本电脑、带用户界面的 PLC），系统集成商必须满足以下要求：

- 系统集成商必须确保整个系统满足 ISO 10218-2 标准中关于单点控制的要求。
- 系统集成商必须在风险评估中考虑到，在此应用场景下，KUKA 控制器不提供用于释放外部控制的安全功能（其性能等级为 a）。

## 违规使用

所有与规定用途不符的使用方式都属于违规使用并且均被禁止。对于由于违规使用而造成的损失，制造商不承担任何责任。相关风险完全由使用方承担。

违规使用例如包括：

- 使用备选软件包通过外部操作设备控制机器人，但未遵守 ISO 10218-2 标准中关于单点控制的要求
- 基于以太网的通信不能完全替代现场总线系统。它不适用于操作外部自动运行接口。
- 备选软件包不保证确定性的时间行为。因此，所获取的数据可能以不同的速度提供。
- 备选软件包不适用于建立与机器人控制器实时节拍同步的周期性通信。

# 3 安全

有关工业机器人的安全信息，参见系统软件操作与编程指南中的“安全”一章。

**注意与安全相关的信息**

安全使用本产品需要掌握知识和遵守基本安全措施。否则有可能造成人员死亡、重伤或财产损失。

- 必须注意遵守系统软件操作和编程指南中的“安全”一章。

# 4 网络安全

本文档包含专门针对本文件所述备选软件包的网络安全提示。

关于网络安全的基本信息，请参阅系统软件操作与编程指南中的“网络安全”一章。

**请遵守网络安全相关信息**

只允许在技术情况完好的状态下按规定且有安全意识地使用 KUKA 产品。有安全意识地使用尤其意味着，产品必须在符合当前网络安全标准且基于整体网络安全概念的 IT 环境中运行。

- 必须注意遵守系统软件操作与编程指南中的“网络安全”一章。

## 4.1 网络安全提示

以下说明描述了在使用 iiQKA.EthernetKRL (EKI) 备选软件包时的安全相关方面。为确保 EKI 接口的安全运行，用户必须采取适当的组织和技术措施。

### 身份验证与授权

该备选软件包目前不提供任何集成的身份验证或授权功能。

通过 EKI 接口进行数据交换时，存在以下安全相关风险：

- 未加密的数据传输
- 无身份验证或授权机制
- 无法自动将外部连接限制为可信来源

因此，建议仅在受保护的网络环境中运行 EKI 接口，并在网络层面实施适当的防护措施。

### 传入数据的验证

建议对通过 EKI 接口接收到的传入数据，在进行后续处理之前进行仔细的检查和验证。

备选软件包仅保证所接收数据的类型安全性。更深入的内容检查、合理性检查或数值检查必须由用户自行实现。

### 加密

该备选软件包目前不支持加密通信。因此，数据传输必须在网络层面加以保护。

# 5 安装

## 5.1 系统要求

### 硬件

- 机器人控制器 KR C5-2 或 KR C5 micro-2
- 笔记本电脑/计算机

### 软件

**在机器人控制器上：**

- iiQKA.OS2 9.x（自 9.3 版本起）

**在笔记本电脑/计算机上：**

- iiQWorks.Sim 1.x（自 1.3 版本起）
  备选软件包（包含在 iiQKA.OS2 的供货范围内）：
  - iiQKA.CommonApplicationMessage
- iiQWorks.App Builder 1.x（自 1.2 版本起）

## 5.2 安装或更新 iiQKA.EthernetKRL

### 说明

该备选软件包可以通过 iiQWorks.Sim 安装到机器人控制器中。

| | |
|---|---|
| i | 更新时将自动采用现有配置。如果此非所愿，则必须先卸载现有版本。 |
| i | 建议在更新软件之前存档所有相关数据。 |

### 前提条件

在 iiQWorks.Sim 中：

- 与机器人控制器建立网络连接

在机器人控制器上：

- SPOC
- 专家或更高用户组
- T1 或 T2 运行模式
- 未选择程序。

### 操作步骤

1. 将 iiQKA.EthernetKRL 备选软件包安装在 iiQWorks.Sim 中。
2. 从机器人控制器载入激活的实例配置。
   或者创建新的实例配置，并将所用机器人拖放到 3D 视图中。
3. 将 iiQKA.EthernetKRL 备选软件包分配给机器人控制器。
4. 配置备选软件包。
   (>>> *7.1 "概览：投入运行和配置" 页面 23*)
5. 将实例配置从 iiQWorks.Sim 传输到机器人控制器并激活。
   该备选软件包即被安装。

| | |
|---|---|
| i | 关于 iiQWorks.Sim 中流程步骤的信息参见 iiQWorks.Sim 的文档。 |

## 5.3 iiQKA.EthernetKRL 卸载

### 说明

该备选软件包可以通过 iiQWorks.Sim 从机器人控制器中卸载。

| | |
|---|---|
|  | 我们建议在软件卸载前将所有相关数据存档。 |

### 前提条件

在 iiQWorks.Sim 中：

- 与机器人控制器建立网络连接

在机器人控制器上：

- SPOC
- 专家或更高用户组
- T1 或 T2 运行模式
- 未选择程序。

### 操作步骤

1. 从机器人控制器载入激活的实例配置。
2. 在实例配置中，将 iiQKA.EthernetKRL 备选软件包从机器人控制器中删除。
3. 将实例配置从 iiQWorks.Sim 传输到机器人控制器并激活。
   备选软件包即被卸载。

| | |
|---|---|
|  | 关于 iiQWorks.Sim 中流程步骤的信息参见 iiQWorks.Sim 的文档。 |

# 6 操作

## 6.1 管理以太网配置

### 说明

以太网配置可以在 iiQWorks.Sim 中进行管理，但无法创建或编辑。
如需创建和编辑配置，可使用 iiQWorks.App Builder 中提供的图形化编辑器。
在 iiQWorks.Sim 中提供以下功能：

- 导出、导入和删除以太网配置
- 复制和重命名以太网配置
- 更改以太网配置的 IP 地址和端口

### 前提条件

- 3D 视图中有机器人
- 备选软件包 iiQKA.EthernetKRL 已分配给机器人控制器
- **开始**或**配置**功能区

### 操作步骤

1. 在**设备**窗口中展开机器人控制器的节点。
2. 依次选择节点：**备选软件包** > **iiQKA.EthernetKRL** > **以太网配置**。
3. 在**属性**窗口中将显示所有现有的以太网配置。
4. 如需重命名，请在设备树中选中相应的以太网配置，随后在**属性**窗口中输入新名称。

### 概览

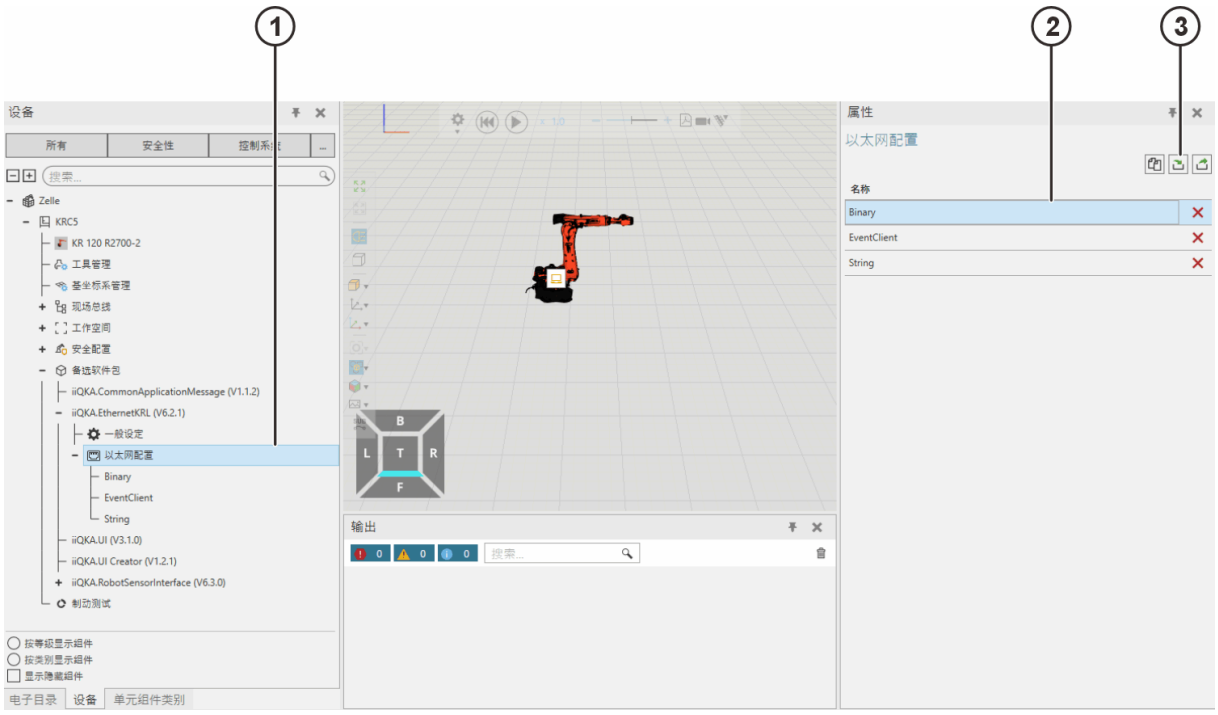



图 6-1: 以太网配置

1 设备树中的以太网配置
2 属性中的以太网配置

3 操作栏

**操作栏**

可提供以下操作：

| 按钮 | 说明 |
|---|---|
|  | **复制配置** |
|  | **导入配置** |
|  | **导出配置** |
|  | **删除配置** |

## 6.2 更改以太网配置的 IP 地址或端口

**说明**

现有以太网配置的 IP 地址和/或端口号可以直接在 iiQWorks.Sim 中调整。为此无需在 iiQWorks.App Builder 中编辑以太网配置。

**前提条件**

- 3D 视图中有机器人
- 备选软件包 iiQKA.EthernetKRL 已分配给机器人控制器
- **开始**或**配置**功能区

**操作步骤**

1. 在**设备**窗口中展开机器人控制器的节点。
2. 导航至**备选软件包** > **iiQKA.EthernetKRL** > **以太网配置** 并选择所需的配置。
3. 在**属性**窗口中，根据需要对 IP 地址和/或端口号进行调整。
4. 保存更改。

# 7 投入运行和配置

## 7.1 概览：投入运行和配置

关于 iiQWorks.Sim 中流程步骤的信息参见 iiQWorks.Sim 的文档。

关于 iiQWorks.App Builder 中流程步骤的信息参见 iiQWorks.App Builder 的文档。

| 步骤 | 说明 |
|---|---|
| 1 | 将 iiQKA.EthernetKRL 备选软件包安装在 iiQWorks.Sim 中。 |
| 2 | 从机器人控制器载入激活的实例配置。<br>或者创建新的实例配置，并将所用机器人拖放到 3D 视图中。 |
| 3 | 将 iiQKA.EthernetKRL 备选软件包分配给机器人控制器。 |
| 4 | 通过 KLI 配置网络连接<br>(>>> *7.2 "通过 KLI 建立网络连接" 页面 23*) |
| 5 | 使用 iiQWorks.App Builder 创建和编辑以太网配置<br>(>>> *7.3 "创建和编辑以太网配置" 页面 23*) |
| 6 | 可选：将 iiQWorks.App Builder 中的以太网配置预先部署至虚拟机器人控制器以进行测试<br>(>>> *7.4 "将以太网配置传输到机器人控制器" 页面 32*) |
| 7 | 将 iiQWorks.App Builder 中的以太网配置导出，并导入到 iiQWorks.Sim 中。 |
| 8 | 将实例配置从 iiQWorks.Sim 传输到机器人控制器并激活。 |

## 7.2 通过 KLI 建立网络连接

为了通过以太网进行数据交换，需要通过机器人控制器的 KLI 建立网络连接。

有关 KLI 网络配置的详细信息参见系统软件的操作与编程指南。

可用的以太网接口详细信息请见机器人控制系统的安装指南。

## 7.3 创建和编辑以太网配置

### 概览

在 iiQWorks.App Builder 中提供以下功能：

- 创建和编辑新的以太网配置（图形化编辑器）
  每个以太网连接（通道）都必须创建一份独立的以太网配置。以太网配置的名称同时作为 KRL 中的访问密钥。
  示例：
  - 以太网配置名称：EKISample
  - 在 KRL 中调用：RET = EKI_Open("EKISample")
- 导入现有以太网配置
- 导出、删除及重命名以太网配置

- 将以太网配置传输至机器人控制器
  (>>> *7.4 "将以太网配置传输到机器人控制器" 页面 32*)

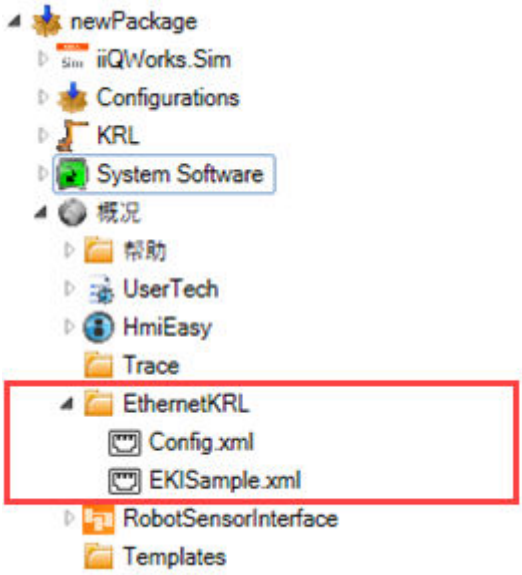



**图 7-1: 以太网配置**

**操作步骤**

**新建文件或导入已有文件:**

1. 右键点击 **EthernetKRL** 文件夹。
2. 在上下文菜单中选择所需的操作。
   新创建或导入的文件将被添加到项目中，并自动在编辑器中打开。

**在编辑器中打开、导出、删除或重命名文件:**

1. 在 **EthernetKRL** 文件夹中右键点击所需文件。
2. 在上下文菜单中选择所需的操作。
   - 或者，也可以通过双击文件在编辑器中将其打开。

**编辑器**

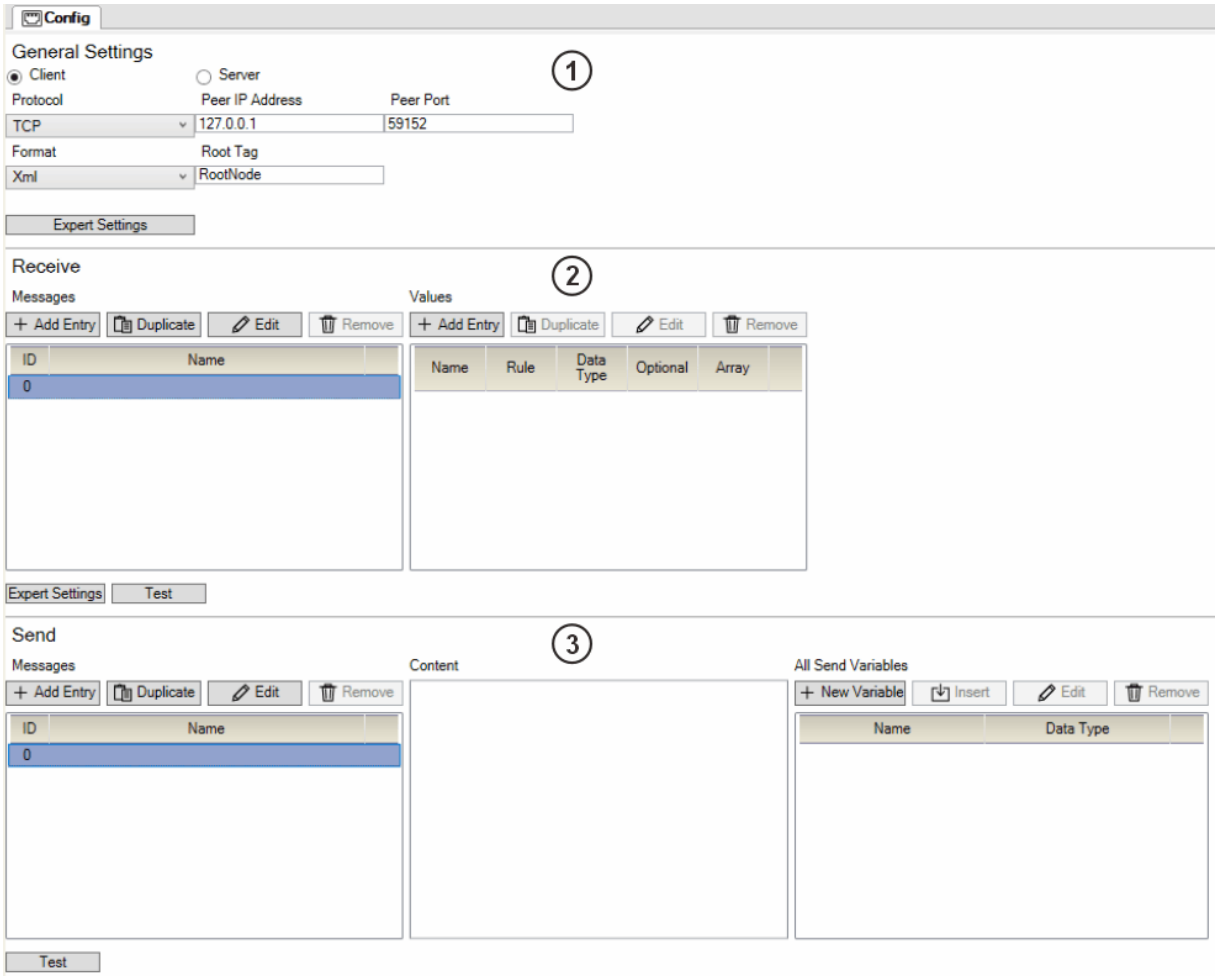



**图 7-2: 编辑器概述**

1 通用设置
2 接收配置
3 发送配置

## 7.3.1 一般设定

**说明**

在此区域中可以配置以太网连接（通道）的属性以及通过该通道交换的消息格式。

**连接参数**

连接参数取决于 EKI 是作为客户端还是作为服务器运行。
EKI 客户端参数：

| 参数 | 说明 |
|---|---|
| 协议 | 选择 IP 通信协议<br>• TCP<br>• UDP |
| 对端 IP 地址 | EKI 客户端所要连接的服务器的 IP 地址<br>• 0.0.0.0 … 255.255.255.255 |

| 参数 | 说明 |
|---|---|
| **对端的端口** | EKI 客户端所要连接的服务器的端口号<br>• 1 … 65535 |

EKI 服务器参数：

| 参数 | 说明 |
|---|---|
| **协议** | 选择 IP 通信协议<br>• TCP<br>• UDP |
| **本地端口** | EKI 服务器等待客户端连接请求的端口号<br>• 54600 … 54615 |

**格式**

当原始数据到达接口时，系统会检查其中是否包含一条完整的消息。此处定义如何从原始数据中提取此类消息。

| 参数 | 说明 |
|---|---|
| **格式** | 选择通过通道交换的消息格式<br>• Xml<br>• String |
| **根节点** | 对于 XML 格式：指定 XML 结构的根节点 |
| **分隔符** | 对于 String 格式：选择并定义分隔符 (Separator)<br>• FixedLength：固定长度消息（最多 4000 个字符）：指定长度<br>• Characters：带有定义结束字符串的消息：指定结束字符串<br>• Custom：带有可变字符串的自定义消息：指定正则表达式 (Regex) |

**专家设置**

在专家设置中可以配置以太网连接的以下属性：

- **缓冲**（可选）
  通过这些设置可以更改接收缓冲区的属性。

| 参数 | 说明 |
|---|---|
| **缓冲** | 选择从缓冲区读取消息的方式<br>– FIFO：First In First Out（先进先出）<br>最先接收到的消息最先被提取。<br>– LIFO：后进先出（Last In First Out）<br>最后接收到的消息最先被提取。 |
| **数量** | 可存储的最大消息数量<br>– 1 … 512 |
| **策略** | 选择接收缓冲区已满时的策略<br>– Error：通信通道将被关闭。可在程序中使用 EKI_Open 函数重新打开。<br>– DiscardNew：丢弃所有新到达的消息。<br>– DiscardOldest：丢弃最早接收到的消息。 |

- **通信错误时的事件**（可选）
  为了报告通信错误，可以定义一个数字输出 ($OUT) 或一个标志 ($FLAG)。
  - 当通过 EKI_Open 打开通信通道时，配置的输出或标志会自动设置为 FALSE。
  - 如果发生通信错误，输出或标志将被设置为 TRUE。
  - 执行 EKI_Load 和 EKI_Open 函数后，输出或标志将再次重置为 FALSE。

| 参数 | 说明 |
|---|---|
| **通信错误时的事件** | 启用此选项以定义该事件 |
| **事件类型** | 选择事件类型<br>– **Flag**：标志 $FLAG<br>– **Out**：数字输出 $OUT |
| **索引** | 要使用的输出或标志的编号 |

- **Ping**（可选）
  通过 Ping 进行连接监控：可以按固定时间间隔向对端发送 Ping 消息。
- **使用备用的对端 IP**（可选）
  仅适用于 EKI 客户端：当 EKI 在虚拟机器人控制器 (VRC) 中运行时，客户端可以连接至一个备用 IP 地址。这样就可以在不改变真实机器人控制器现有配置的情况下进行仿真测试。
- **通过解释器复位进行删除**（可选）
  当所设置的解释器被取消选择或重置时，以太网连接及接收缓冲区中的所有数据将被自动删除。默认情况下，这一操作在机器人解释器重置时执行。
  可用解释器：
  - **R_INT**：机器人解释器（默认设置）
  - **S_INT**：系统提交解释器
  - **S_USER1 … S_USER7**：扩展提交解释器
  - **ANY**：在任何解释器上都会删除连接。
  - **NONE**：在任何解释器上都不会删除连接。

## 7.3.2 接收配置

### 说明

在接收配置中可以创建和管理不同的消息变体。此外，此处还定义了如何从传入消息中提取与解析数据的具体规则。

### 消息

通过以太网连接（通道）可以接收同一消息的不同变体。在此区域中创建消息变体。

- 每条消息都有一个唯一的 ID。在 KRL 程序中可通过该 ID 访问对应的消息。
  (>>> *8.2.6 "EKI_ReadNext：选择下一个数据包" 页面 38*)
- 此外，还可为每条消息分配一个自定义名称。该名称对机器人程序没有影响。它仅用于帮助用户在编辑器中更好地定位和识别。

当接收到一条消息时，EKI 会判断其属于哪种消息变体。判定标准为：必须能够找到该变体所包含的所有必填值（即非可选值）。如果多个变体都满足此条件，则首先比较必填值的数量。若仍无法明确归属，则将选择 ID 数值最小的变体。

### 值

每种消息变体均包含一组独立的值。在此区域中可以创建和定义这些值。

- 每个值都有一个唯一的名称。在 KRL 程序中可通过该名称访问对应的值。
- 为每个值定义一条规则。该规则决定了如何从消息中提取和解释该值。
- 根据通道所接收消息的格式，规则的指定方式也有所不同。
- 接收消息中的值可被标记为“可选”。
  - 可选值在消息中不是必须存在的，消息仍可视为完整。
  - 使用 KRL 函数 EKI_HasValue 可以检查接收到的消息中是否存在标记为可选的值。
    (>>> *8.2.30 "EKI_HasValue：检查可选值是否存在" 页面 55*)
- 参数 **数组** 参数定义了当一条规则匹配到多个值时的处理行为：
  - 如果参数 **数组** 未激活，当匹配到多个值时，则仅提取并保存第一个值。在 KRL 中可以通过 EKI_Get 函数读取该值。
  - 如果参数 **数组** 激活，则提取并保存所有接收到的值。在 KRL 中只能通过 EKI_Get…Array 函数访问这些数值。

#### XML 格式

以下参数用于定义 XML 格式消息中的值：

| 参数 | 说明 |
|---|---|
| **名称** | 用于存储值的唯一名称<br>该名称后续将用作在 KRL 中进行访问的 ID。 |
| **XPath** | XML 消息中值的路径<br>• 如果 XPath 表达式在消息中找到了多个匹配项，则使用第一个匹配项。<br>• 如果将变量配置为数组，则所有匹配项都将被读入该数组。 |
| **数据类型** | 选择用于存储值的数据类型<br>• Bool<br>• Int<br>• Real<br>• String<br>该数据类型同时决定了在 KRL 中读取值时的类型。 |
| **可选** | 指定值是否必须包含在消息中<br>可选值（勾选状态）允许缺失。 |
| **数组** | 指定是否将规则匹配到的所有值读入一个数组中。 |

#### 字符串格式

以下参数用于定义字符串格式消息中的值：

| 参数 | 说明 |
|---|---|
| **名称** | 用于存储值的唯一名称<br>该名称后续将用作在 KRL 中进行访问的 ID。 |
| **规则** | 用于从字符串中提取值的规则<br>(>>> *"字符串格式规则" 页面 29*) |
| **数据类型** | 选择用于存储值的数据类型<br>• Bool<br>• Int<br>• Real<br>• String<br>该数据类型同时决定了在 KRL 中读取值时的类型。 |
| **可选** | 指定值是否必须包含在消息中<br>可选值（勾选状态）允许缺失。 |
| **数组** | 指定是否将规则匹配到的所有值读入一个数组中。 |

### 字符串格式规则

字符串格式的值始终通过正则表达式 (Regex) 来提取。对于常见的应用场景，系统提供了预定义的规则。或者，也可以指定自定义的正则表达式。

- 如果正则表达式在消息中找到了多个匹配项，则使用第一个匹配项。
- 如果将变量配置为数组，则所有匹配项都将被读入该数组。

输入的参数值会被直接插入到预定义的正则表达式中。因此，在使用正则表达式特殊字符（例如 $ 或 .）时，可能会产生意外的结果。
此类特殊字符必须要么使用 ASCII 码表示，要么使用反斜杠 (\) 进行转义（例如 \$ 或 \.）。

| 规则 | 说明 | 应用场景/示例 |
|---|---|---|
| **所有字符** | 将整个字符串作为值直接采用。 | 不对消息进行进一步解析。 |
| **分隔符** | 从字符串中提取一个值，该字符串中的各个值由一个指定的字符分隔。<br>参数：<br>• **分隔符**<br>字符或字符串<br>• **位置**<br>待读取值的位置 | 消息中包含多个由指定分隔符（例如分号、逗号或字符串）分隔的值。例如 CSV 字符串。<br>示例：<br>• 接收到的消息：<br>"10;True;12.3;MoveInX"<br>• 待读取值的定义：<br>– **名称**： MyValue<br>– **规则**：**分隔符**<br>– **分隔符**： ;<br>– **位置**：3<br>– **数据类型**: Real<br>• 通过上述设置，第三个值“12.3”将被解析为实数值并以名称“MyValue”存储。 |

| 规则 | 说明 | 应用场景/示例 |
| --- | --- | --- |
| **偏移** | 从指定位置开始提取一个固定长度的值。<br>参数：<br>• **偏移**<br>消息中开始读取值的位置<br>• **长度**<br>字符数量 | 解析消息中始终出现在同一位置且具有固定长度的值。例如，二进制数据组中的固定长度数据包。<br>示例：<br>• 接收到的消息：<br>"10;True;12.3;MoveInX"<br>• 待读取值的定义：<br>– **名称**：MyValue<br>– **规则**：**偏移**<br>– **偏移**：0<br>– **长度**：2<br>– **数据类型**：**Int**<br>• 通过上述设置，前两个字符将被解析为整数值并以名称“MyValue”存储。 |
| **内部** | 提取一个位于两个已定义的字符或字符串之间的值。<br>参数：<br>• **启动**<br>标记值开始的字符或字符串<br>也可为空：如果为空，则值从消息的开头开始。<br>• **结束**<br>标记值结束的字符或字符串<br>也可为空：如果为空，则值到消息的末尾结束。<br>如果两个参数都为空，则使用整个字符串。 | 数据以键值对结构存在的消息<br>示例：<br>• 接收到的消息：<br>"a=1.1; b=13.1; c=90.3; x=10.0; y=2.1; z=12.6;"<br>• 待读取值的定义：<br>– **名称**：PosX<br>– **规则**：**内部**<br>– **启动**：x=<br>– **结束**：;<br>– **数据类型**：**Real**<br>• 通过上述设置，将提取位于“x=”和“;”之间的值“10.0”，并将其作为实数值以名称“PosX”存储。 |
| **用户自定义** | 用户自定义规则（自定义正则表达式）<br>参数：<br>• **正则表达式**<br>正则表达式<br>**注意**：使用捕获组时，系统仅处理捕获组的结果。 | 复杂消息结构<br>捕获组使用示例：<br>• 接收到的消息：<br>“My KukaRobot”<br>• 配置的正则表达式：<br>(\w+)Robot<br>• 结果：<br>“Kuka” |

**专家设置**

在专家设置中，可以为数据接收配置以下属性：

- **接收事件**（可选）
  为了报告完整消息的接收情况，可以定义一个数字输出 ($OUT) 或一个标志 ($FLAG)。
  – 当通过 EKI_Open 打开通信通道时，配置的输出或标志会自动设置为 FALSE。

- 当一条完整的消息到达时，输出或标志将被设置为 TRUE。
- 在执行以下函数后，输出或标志将被重置为 FALSE：
  - EKI_Load
  - EKI_ReadNext
  - EKI_Unload
  - EKI_DeleteReceivedData

| 参数 | 说明 |
|---|---|
| **接收事件** | 启用此选项以定义该事件 |
| **事件类型** | 选择事件类型<br>– **Flag**：标志 $FLAG<br>– **Out**：数字输出 $OUT |
| **索引** | 要使用的输出或标志的编号 |

#### 7.3.2.1 测试接收配置

**说明**

可以检查传入数据是否按预期被处理。

**操作步骤**

1. 选择 **测试** 按钮。一个对话框打开。
2. 输入测试数据。系统会在输入内容中搜索有效的完整消息。所有找到的消息及其提取出的值将一并显示。

### 7.3.3 发送配置

**说明**

在发送配置中可以创建和管理不同的消息变体及发送变量。此外，此处还定义了待传输消息的结构。

**消息**

通过以太网连接（通道）可以发送同一消息的不同变体。在此区域中创建消息变体。

- 每条消息都有一个唯一的 ID。在 KRL 程序中可通过该 ID 访问对应的消息。
  (>>> *8.2.17 "EKI_Send：发送数据" 页面 47*)
- 此外，还可为每条消息分配一个自定义名称。该名称对机器人程序没有影响。它仅用于帮助用户在编辑器中更好地定位和识别。

**所有发送变量**

在此区域中定义所有可在消息变体中使用的变量。

- 每个变量都有一个唯一的名称，在 KRL 中可通过该名称对其进行访问。
- 为每个变量指定一个数据类型：
  - Bool
  - Int
  - Real
  - String

  该数据类型同时决定了在 KRL 中设置变量时的类型。

- 一个变量可以在任意数量的消息变体中使用。
- 对变量的修改或删除将影响所有使用该变量的消息变体。

**内容**

在此编辑器中可以创建消息变体的具体内容。

- 通过**新变量**按钮创建的新变量，将自动在当前光标位置插入到所选消息中。
- 已有变量可以通过**粘贴**按钮插入到当前所选消息中。
- 消息文本及变量语法均可在编辑器中手动编辑和调整。

#### 7.3.3.1 测试发送配置

**说明**

可以检查配置是否生成预期的发送消息。

**操作步骤**

1. 选择 **测试** 按钮。将打开一个对话框，并显示所有可用的发送变量。
2. 输入变量值。随后，系统会显示针对对话框中所选消息变体生成的发送消息。

### 7.4 将以太网配置传输到机器人控制器

**说明**

以太网配置通常与来自 iiQWorks.Sim 的完整实例配置一起传输到机器人控制器上。为此，在 iiQWorks.App Builder 中创建的以太网配置必须首先导出，然后再导入到 iiQWorks.Sim 中。

此外，iiQWorks.App Builder 软件还提供了将以太网配置预先部署到虚拟机器人控制器上以供测试的功能。

**操作步骤**

- 选择菜单序列：**附加功能** > **部署到实例…**。

| | |
|---|---|
|  | 有关操作步骤的所有其他信息，请参阅 iiQWorks.App Builder 文档。 |

# 8 编程

在备选软件包 iiQKA.EthernetKRL 6.0 版本中，KRL 函数已被删除，同时新增了 KRL 函数且/或对函数名称进行了修改。使用旧版本备选软件包创建的程序不再兼容。
有关迁移的编程说明请参见附录： (>>> *12.1 "迁移至 iiQKA.EthernetKRL 6.x 的编程说明" 页面 79*)

## 8.1 编程提示

### 并行访问

以太网连接既可以由机器人解释器打开和使用，也可以由提交解释器打开和使用。虽然允许对同一以太网连接或其数据进行并行访问，但这可能会导致错误。

可能引发的问题：

- 在同时访问发送和接收数据时，可能会丢失数据一致性。
- 一个解释器无法感知到另一个解释器可能会打开或关闭连接。
- 当一个解释器被取消选择或重置时，其所占用的以太网连接会被删除，而与此同时另一个解释器可能仍在访问该连接。

建议仅在一个单独的解释器中使用以太网连接：即在机器人解释器、系统提交解释器或扩展提交解释器中任选其一。

## 8.2 KRL 函数

### 说明

为实现数据交换编程，系统提供了专用的 KRL 函数。这些函数中的大多数均返回一个 INT 类型的返回值。因此，用于接收此返回值的变量必须声明为 INT 类型并进行初始化。

该返回值可在 KRL 程序中查询和评估，例如通过 IF-THEN 语句对错误作出响应。

所有可能的返回值均定义为全局常量：

```
GLOBAL CONST INT EkiOk = 0
GLOBAL CONST INT EkiFileNotFound = -1
GLOBAL CONST INT EkiConfigNotFound = -2
GLOBAL CONST INT EkiInvalidArgument = -3
GLOBAL CONST INT EkiUnexpectedError = -4
GLOBAL CONST INT EkiParsingError = -5
GLOBAL CONST INT EkiInvalidOperation = -6
GLOBAL CONST INT EkiNoData = -7
```

以下函数的行为与上述描述不同：

- 函数 EKI_GetBufferedMsgCount 返回的不是错误代码，而是接收缓冲区中的消息数量。
- EKI_Is 函数内部调用 EKI_GetConnectionState 函数并评估其返回值。

## 概览

| 功能 | 说明 |
| --- | --- |
| EKI_Load | 加载以太网配置<br>(>>> *8.2.1 "EKI_Load：加载以太网配置" 页面 35*) |
| EKI_Open | 打开通信通道<br>(>>> *8.2.2 "EKI_Open：打开通信通道" 页面 36*) |
| EKI_Close | 关闭通信通道<br>(>>> *8.2.3 "EKI_Close：关闭通信通道" 页面 37*) |
| EKI_Unload | 卸载以太网配置<br>(>>> *8.2.4 "EKI_Unload：卸载以太网配置" 页面 37*) |
| EKI_DeleteReceivedData | 删除通信通道的所有接收数据<br>(>>> *8.2.5 "EKI_DeleteReceivedData：删除接收到的数据" 页面 38*) |
| EKI_ReadNext | 选择下一个待读取的接收数据包（通过 EKI_Get 函数读取）<br>(>>> *8.2.6 "EKI_ReadNext：选择下一个数据包" 页面 38*) |
| EKI_GetBool | 从选定的接收数据包中读取布尔值<br>(>>> *8.2.7 "EKI_GetBool：读取布尔值" 页面 39*) |
| EKI_GetBoolArray | 从选定的接收数据包中读取布尔值数组<br>(>>> *8.2.8 "EKI_GetBoolArray：读取 BOOL 数组" 页面 39*) |
| EKI_GetInt | 从选定的接收数据包中读取整数值<br>(>>> *8.2.9 "EKI_GetInt：读取整数值" 页面 40*) |
| EKI_GetIntArray | 从选定的接收数据包中读取整数值数组<br>(>>> *8.2.10 "EKI_GetIntArray：读取 INT 数组" 页面 41*) |
| EKI_GetReal | 从选定的接收数据包中读取实数值<br>(>>> *8.2.11 "EKI_GetReal：读取实数值" 页面 42*) |
| EKI_GetRealArray | 从选定的接收数据包中读取实数值数组<br>(>>> *8.2.12 "EKI_GetRealArray：读取 REAL 数组" 页面 43*) |
| EKI_GetString | 从选定的接收数据包中读取字符串<br>(>>> *8.2.13 "EKI_GetString：读取字符串" 页面 44*) |
| EKI_GetStringArray | 从选定的接收数据包中读取字符串值数组<br>(>>> *8.2.14 "EKI_GetStringArray：读取字符串数组" 页面 44*) |
| EKI_GetFrame | 从选定的接收数据包中读取 KRL 坐标系<br>(>>> *8.2.15 "EKI_GetFrame：读取 KRL 坐标系" 页面 45*) |
| EKI_GetFrameArray | 从选定的接收数据包中读取 KRL 坐标系数组<br>(>>> *8.2.16 "EKI_GetFrameArray：读取 KRL 坐标系数组" 页面 46*) |
| EKI_Send | 发送发送缓冲区中的数据<br>(>>> *8.2.17 "EKI_Send：发送数据" 页面 47*) |
| EKI_SetBool | 在发送缓冲区中设置布尔值<br>(>>> *8.2.18 "EKI_SetBool：设置布尔值" 页面 48*) |

| 功能 | 说明 |
| --- | --- |
| EKI_SetInt | 在发送缓冲区中设置整数值<br>(>>> *8.2.19 "EKI_SetInt：设置整数值" 页面 49*) |
| EKI_SetReal | 在发送缓冲区中设置实数值<br>(>>> *8.2.20 "EKI_SetReal：设置实数值" 页面 49*) |
| EKI_SetString | 在发送缓冲区中设置字符串值<br>(>>> *8.2.21 "EKI_SetString：设置字符串" 页面 50*) |
| EKI_SetFrame | 将一个 KRL 坐标系的各分量写入发送缓冲区<br>(>>> *8.2.22 "EKI_SetFrame：设置 KRL 坐标系" 页面 51*) |
| EKI_GetBufferedMsgCount | 返回接收缓冲区中的消息数量<br>(>>> *8.2.23 "EKI_GetBufferedMsgCount：读取已接收消息的数量" 页面 51*) |
| EKI_GetConnectionState | 返回通信通道的连接状态<br>(>>> *8.2.24 "EKI_GetConnectionState：查询连接状态" 页面 52*) |
| EKI_IsClosed | 检查通信通道是否已关闭<br>(>>> *8.2.25 "EKI_IsClosed：检查通道是否已关闭" 页面 53*) |
| EKI_IsWaiting | 检查通信通道是否处于等待状态<br>(>>> *8.2.26 "EKI_IsWaiting：检查通道是否处于等待状态" 页面 53*) |
| EKI_IsActive | 检查通信通道是否处于激活状态<br>(>>> *8.2.27 "EKI_IsActive：检查通道是否处于激活状态" 页面 53*) |
| EKI_IsLoaded | 检查通信通道是否已加载<br>(>>> *8.2.28 "EKI_IsLoaded：检查通道是否已加载" 页面 54*) |
| EKI_IsInState | 检查通信通道是否处于预期状态<br>(>>> *8.2.29 "EKI_IsInState：检查通道是否处于预期状态" 页面 54*) |
| EKI_HasValue | 检查选定的接收数据包中是否包含指定的数值<br>(>>> *8.2.30 "EKI_HasValue：检查可选值是否存在" 页面 55*) |
| EKI_HasFrameValue | 检查选定的接收数据包中是否存在完整的 KRL 坐标系<br>(>>> *8.2.31 "EKI_HasFrameValue：检查是否存在完整的 KRL 坐标系" 页面 56*) |

### 8.2.1 EKI_Load：加载以太网配置

**说明**

该函数用于加载以太网配置。

**声明**

INT EKI_Load(*configName[]:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiFileNotFound | *未找到配置“{0}”。* |
| EkiParsingError | *读取配置“{0}”时发生错误。* |
| EkiInvalidOperation | *配置“{0}”已加载。* |
| | *已加载的配置数量已达上限。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_Load("VisionCom")
```

### 8.2.2 EKI_Open：打开通信通道

**说明**

该函数用于打开通信通道。此时尚未通过该通道交换数据。

- EKI 作为客户端：尝试与对端建立连接。
- EKI 作为服务器：将打开一个端口。此时客户端可开始连接。

**声明**

INT EKI_Open(*configName[]:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *打开连接“{0}”失败。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_Open("VisionCom")
```

### 8.2.3 EKI_Close：关闭通信通道

**说明**

该函数用于关闭通信通道。无法再通过该通道交换任何数据。

**声明**

INT EKI_Close(*configName[]:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_Close("VisionCom")
```

### 8.2.4 EKI_Unload：卸载以太网配置

**说明**

该函数用于卸载以太网配置。

**声明**

INT EKI_Unload(*configName[]:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_Unload("VisionCom")
```

### 8.2.5 EKI_DeleteReceivedData：删除接收到的数据

**说明**

该函数用于删除所有已接收的数据。

**声明**

INT EKI_DeleteReceivedData(*configName[]:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_DeleteReceivedData("VisionCom")
```

### 8.2.6 EKI_ReadNext：选择下一个数据包

**说明**

该函数会选择下一个接收到的数据包。从此时起，EKI_Get 函数将返回所选接收包的值。

**声明**

INT EKI_ReadNext(*configName[]:IN, <msgId:OUT>*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *<msgId:OUT>* | 数据类型：INT<br>可选：返回所接收消息的 ID。 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiNoData | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
DECL INT msgId
; ...
RET = EKI_ReadNext("VisionCom", msgId)
```

### 8.2.7 EKI_GetBool：读取布尔值

**说明**

该函数用于从选定的接收数据包中读取布尔值。

**声明**

INT EKI_GetBool(*configName[]:IN, variableName[]:IN, value:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待读取变量的名称 |
| *value:OUT* | 数据类型：BOOL<br>从选定的接收数据包中获取的变量值 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”的当前数据组不包含类型为“{2}”的元素“{1}”。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_GetBool("VisionCom", "Success", isValid)
```

### 8.2.8 EKI_GetBoolArray：读取 BOOL 数组

**说明**

该函数用于从选定的接收数据包中读取布尔值数组。

**声明**

INT EKI_GetBoolArray(*configName[]:IN, variableName[]:IN, values[]:OUT, iterator:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待读取变量的名称 |
| *values[]:OUT* | 数据类型：BOOL 数组<br>目标数组，接收到的数据将被写入该数组中 |
| *iterator:OUT* | 数据类型：EkiIterator_T<br>用于数据访问的结构体 |

**数据类型 EkiIterator_T**

```
STRUC EkiIterator_T INT Count, Offset, Remaining
```

| 元素 | 说明 |
|---|---|
| Count | 数据类型：INT<br>上次访问时读取的元素数量 |
| Offset | 数据类型： INT (IN-/OUT)<br>在接收到的数据组中读取的起始位置<br>读取完成后，偏移量会根据已读取元素的数量相应增加。若仍存在尚未读取的元素，则可继续使用此迭代器进行读取。 |
| Remaining | 数据类型：INT<br>尚未读取的元素数量 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”的当前数据组不包含类型为“{2}”的元素“{1}”。* |
| EkiInvalidArgument | *读取数组“{0}”时，偏移量 {1} 无效。* |

**示例**

```
RET = EKI_GetBoolArray("VisionCom", "Success", bools[],
iterator)
```

(>>> *8.5 “读取数组数据” 页面 60*)

## 8.2.9 EKI_GetInt：读取整数值

**说明**

该函数用于从选定的接收数据包中读取整数值。

**声明**

INT EKI_GetInt(*configName[]:IN, variableName[]:IN, value:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待读取变量的名称 |
| *value:OUT* | 数据类型：INT<br>从选定的接收数据包中获取的变量值 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”的当前数据组不包含类型为“{2}”的元素“{1}”。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_GetInt("VisionCom", "PartCount", partCounter)
```

## 8.2.10 EKI_GetIntArray：读取 INT 数组

**说明**

该函数用于从选定的接收数据包中读取整数值数组。

**声明**

INT EKI_GetIntArray(*configName[]:IN, variableName[]:IN, values[]:OUT, iterator:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待读取变量的名称 |
| *values[]:OUT* | 数据类型：INT 数组<br>目标数组，接收到的数据将被写入该数组中 |
| *iterator:OUT* | 数据类型：EkiIterator_T<br>用于数据访问的结构体 |

**数据类型 EkiIterator_T**

```
STRUC EkiIterator_T INT Count, Offset, Remaining
```

| 元素 | 说明 |
|---|---|
| Count | 数据类型：INT<br>上次访问时读取的元素数量 |
| Offset | 数据类型： INT (IN-/OUT)<br>在接收到的数据组中读取的起始位置<br>读取完成后，偏移量会根据已读取元素的数量相应增加。若仍存在尚未读取的元素，则可继续使用此迭代器进行读取。 |
| Remaining | 数据类型：INT<br>尚未读取的元素数量 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”的当前数据组不包含类型为“{2}”的元素“{1}”。* |
| EkiInvalidArgument | *读取数组“{0}”时，偏移量 {1} 无效。* |

**示例**

```
RET = EKI_GetIntArray("VisionCom", "Success", ints[],
iterator)
```

(>>> *8.5 "读取数组数据" 页面 60*)

### 8.2.11 EKI_GetReal：读取实数值

**说明**

该函数用于从选定的接收数据包中读取实数值。

**声明**

INT EKI_GetReal(*configName[]:IN, variableName[]:IN, value:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待读取变量的名称 |
| *value:OUT* | 数据类型：REAL<br>从选定的接收数据包中获取的变量值 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |

| 常量 | 信息 |
|---|---|
| EkiInvalidOperation | *配置“{0}”的当前数据组不包含类型为“{2}”的元素“{1}”。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_GetReal("VisionCom", "XPos", xPos)
```

## 8.2.12 EKI_GetRealArray：读取 REAL 数组

**说明**

该函数用于从选定的接收数据包中读取实数值数组。

**声明**

INT EKI_GetRealArray(*configName[]:IN, variableName[]:IN, values[]:OUT, iterator:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待读取变量的名称 |
| *values[]:OUT* | 数据类型：REAL 数组<br>目标数组，接收到的数据将被写入该数组中 |
| *iterator:OUT* | 数据类型：EkiIterator_T<br>用于数据访问的结构体 |

**数据类型 EkiIterator_T**

STRUC EkiIterator_T INT Count, Offset, Remaining

| 元素 | 说明 |
|---|---|
| Count | 数据类型：INT<br>上次访问时读取的元素数量 |
| Offset | 数据类型： INT (IN-/OUT)<br>在接收到的数据组中读取的起始位置<br>读取完成后，偏移量会根据已读取元素的数量相应增加。若仍存在尚未读取的元素，则可继续使用此迭代器进行读取。 |
| Remaining | 数据类型：INT<br>尚未读取的元素数量 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |

| 常量 | 信息 |
|---|---|
| EkiInvalidOperation | *配置“{0}”的当前数据组不包含类型为“{2}”的元素“{1}”。* |
| EkiInvalidArgument | *读取数组“{0}”时，偏移量 {1} 无效。* |

**示例**

```
RET = EKI_GetRealArray("VisionCom", "Success", reals[], iterator)
```

(>>> *8.5 “读取数组数据” 页面 60*)

### 8.2.13 EKI_GetString：读取字符串

**说明**

该函数用于从选定的接收数据包中读取字符串。

**声明**

INT EKI_GetString(*configName[]:IN, variableName[]:IN, value[]:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待读取变量的名称 |
| *value[]:OUT* | 数据类型：CHAR<br>从选定的接收数据包中获取的变量值 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”的当前数据组不包含类型为“{2}”的元素“{1}”。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_GetString("VisionCom", "Msg", peerMsg[])
```

### 8.2.14 EKI_GetStringArray：读取字符串数组

**说明**

该函数用于从选定的接收数据包中读取字符串值列表（数组）。

**声明**

INT EKI_GetStringArray(*configName[]:IN, variableName[]:IN, values[ , ]:OUT, iterator:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待读取变量的名称 |
| *values[ , ]:OUT* | 数据类型： CHAR[ , ]<br>目标数组，接收到的数据将被写入该数组中 |
| *iterator:OUT* | 数据类型：EkiIterator_T<br>用于数据访问的结构体 |

**数据类型 EkiIterator_T**

STRUC EkiIterator_T INT Count, Offset, Remaining

| 元素 | 说明 |
|---|---|
| Count | 数据类型：INT<br>上次访问时读取的元素数量 |
| Offset | 数据类型： INT (IN-/OUT)<br>在接收到的数据组中读取的起始位置<br>读取完成后，偏移量会根据已读取元素的数量相应增加。若仍存在尚未读取的元素，则可继续使用此迭代器进行读取。 |
| Remaining | 数据类型：INT<br>尚未读取的元素数量 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”的当前数据组不包含类型为“{2}”的元素“{1}”。* |
| EkiInvalidArgument | *读取数组“{0}”时，偏移量 {1} 无效。* |

**示例**

```
RET = EKI_GetStringArray("VisionCom", "Success",
strings[ , ], iterator)
```

(>>> *8.5 “读取数组数据” 页面 60*)

### 8.2.15 EKI_GetFrame: 读取 KRL 坐标系

**说明**

该函数用于从选定的接收数据包中读取 KRL 坐标系。

一个 KRL 坐标系由六个实数值（X、Y、Z、A、B、C）组成，这些值共同组织在一个坐标系名称之下。读取时只需指定坐标系名称即可。

**声明**

INT EKI_GetFrame(*configName[]:IN, frameName[]:IN, value:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *frameName[]:IN* | 数据类型：CHAR<br>待读取坐标系的名称 |
| *value:OUT* | 数据类型：FRAME<br>从选定的接收数据包中获取的变量值 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”的当前数据组不包含类型为“{2}”的元素“{1}”。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_GetFrame("VisionCom", "Position", frame)
```

## 8.2.16 EKI_GetFrameArray：读取 KRL 坐标系数组

**说明**

该函数用于从选定的接收数据包中读取 KRL 坐标系数组。

一个 KRL 坐标系由六个实数值（X、Y、Z、A、B、C）组成，这些值共同组织在一个坐标系名称之下。读取时只需指定坐标系名称以及坐标系数组的名称即可。

**声明**

INT EKI_GetFrameArray(*configName[]:IN, frameName[]:IN, values[]:OUT, iterator:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *frameName[]:IN* | 数据类型：CHAR<br>待读取坐标系的名称 |
| *values[]:OUT* | 数据类型：FRAME 数组<br>目标数组，接收到的数据将被写入该数组中 |

| 参数 | 说明 |
|---|---|
| *iterator:OUT* | 数据类型： EkiIterator_T<br>用于数据访问的结构体 |

**数据类型 EkiIterator_T**

```
STRUC EkiIterator_T INT Count, Offset, Remaining
```

| 元素 | 说明 |
|---|---|
| Count | 数据类型：INT<br>上次访问时读取的元素数量 |
| Offset | 数据类型： INT (IN-/OUT)<br>在接收到的数据组中读取的起始位置<br>读取完成后，偏移量会根据已读取元素的数量相应增加。若仍存在尚未读取的元素，则可继续使用此迭代器进行读取。 |
| Remaining | 数据类型：INT<br>尚未读取的元素数量 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”的当前数据组不包含类型为“{2}”的元素“{1}”。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |
| EkiParsingError | *因数据不一致，读取数组“{0}”失败。* |

**示例**

```
RET = EKI_GetFrameArray("VisionCom", "Position", frames[], iterator)
```

(>>> *8.5 “读取数组数据” 页面 60*)

## 8.2.17 EKI_Send：发送数据

**说明**

该函数将发送缓冲区中的数据插入到待发送的消息中。随后，消息将被发送至对端。

**声明**

INT EKI_Send(*configName[]:IN*, <*msgId:IN*>)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *<msgId:IN>* | 数据类型：INT<br>可选：发送一个特定的消息变体。如果未传递 ID，则发送 ID 为 0 的消息。 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *通过“{0}”发送数据失败。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_Send("VisionCom", 5)
```

## 8.2.18 EKI_SetBool：设置布尔值

**说明**

该函数用于将布尔值写入发送缓冲区。

**声明**

INT EKI_SetBool(*configName[]:IN, variableName[]:IN, value:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待写入变量的名称 |
| *value:IN* | 数据类型：BOOL<br>变量值 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”不包含类型为“{2}”的发送元素“{1}”。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_SetBool("VisionCom", "Trigger", true)
```

### 8.2.19 EKI_SetInt：设置整数值

**说明**

该函数用于将整数值写入发送缓冲区。

**声明**

INT EKI_SetInt(*configName[]:IN, variableName[]:IN, value:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待写入变量的名称 |
| *value:IN* | 数据类型：INT<br>变量值 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”不包含类型为“{2}”的发送元素“{1}”。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_SetInt("VisionCom", "TaskID", 5)
```

### 8.2.20 EKI_SetReal：设置实数值

**说明**

该函数用于将实数值写入发送发送缓冲区。

**声明**

INT EKI_SetReal(*configName[]:IN, variableName[]:IN, value:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待写入变量的名称 |
| *value:IN* | 数据类型：REAL<br>变量值 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”不包含类型为“{2}”的发送元素“{1}”。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_SetReal("VisionCom", "Threshold", 99.7)
```

## 8.2.21 EKI_SetString：设置字符串

**说明**

该函数用于将字符串写入发送缓冲区。

**声明**

INT EKI_SetString(*configName[]:IN, variableName[]:IN, value[]:IN, <length:IN>*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *variableName[]:IN* | 数据类型：CHAR<br>待写入变量的名称 |
| *value[]:IN* | 数据类型：CHAR<br>变量值 |
| *<length:IN>* | 数据类型：INT<br>可选：指定应使用 *value[]* 中的多少个字符。<br>如果未指定该参数，则 CHAR 数组将被解释为以空字符结尾的字符串。 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”不包含类型为“{2}”的发送元素“{1}”。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_SetString("VisionCom", "JobName", "FindSomething")
```

## 8.2.22 EKI_SetFrame：设置 KRL 坐标系

### 说明

该函数用于将一个 KRL 坐标系的各分量写入发送缓冲区。

一个 KRL 坐标系由六个实数值（X、Y、Z、A、B、C）组成，这些值共同组织在一个坐标系名称之下。

### 声明

INT EKI_SetFrame(*configName[]:IN, frameName[]:IN, values:IN*)

### 参数

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *frameName[]:IN* | 数据类型：CHAR<br>待写入坐标系的名称 |
| *values:IN* | 数据类型：FRAME<br>坐标系的值 |

### 返回值 (RET)

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiInvalidOperation | *配置“{0}”不包含类型为“{2}”的发送元素“{1}”。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

### 示例

```
RET = EKI_SetFrame("VisionCom", "Position", frame)
```

## 8.2.23 EKI_GetBufferedMsgCount：读取已接收消息的数量

### 说明

该函数返回接收缓冲区中的消息数量。通过 EKI_ReadNext 选择读取的消息不再属于缓冲数据的一部分。

### 声明

INT EKI_GetBufferedMsgCount(*configName[]:IN*)

### 参数

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |

### 返回值

值 < 0：读取时出错

值 >= 0：接收缓冲区中的消息数量

**示例**

```
counter = EKI_GetBufferedMsgCount("VisionCom")
```

## 8.2.24 EKI_GetConnectionState：查询连接状态

**说明**

该函数用于返回通信通道的连接状态。可以查询是否能够与对端进行通信。

**声明**

INT EKI_GetConnectionState(*configName[]:IN, value:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *value:OUT* | 数据类型： ENUM EkiConnectionStates<br>通信通道的连接状态 |

**数据类型 EkiConnectionStates**

```
ENUM EkiConnectionStates Closed, Waiting, Active,
NotLoaded, Unknown
```

| 状态 | 说明 |
|---|---|
| #Closed | 连接已关闭，无法与对端进行通信。<br>该状态出现在以下事件之后：<br>• EKI_Load<br>• EKI_Close<br>• 发生错误，例如通信故障 |
| #Waiting | EKI 已为通信做好准备，但仍在等待外部边界条件。此时无法与对端通信。<br>仅适用于 EKI 服务器：在执行 EKI_Open 后，等待对端（即发起连接的客户端）接入。 |
| #Active | 与对端已成功建立连接。此时可以正常发送和接收数据。 |
| #NotLoaded | 请求的通信通道尚未被加载。 |
| #Unknown | 连接状态未知，因为无法进行查询。 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
DECL EkiConnectionStates state
RET = EKI_GetConnectionState("VisionCom", state)
```

### 8.2.25 EKI_IsClosed：检查通道是否已关闭

**说明**

该函数用于检查通信通道是否已关闭（= 连接状态 #Closed）。

**声明**

BOOL EKI_IsClosed(*configName[]:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |

**示例**

```
closed = EKI_IsClosed("VisionCom")
```

### 8.2.26 EKI_IsWaiting：检查通道是否处于等待状态

**说明**

该函数用于检查通信通道是否处于等待状态（即 EKI 服务器在调用 EKI_Open 后的连接状态 #Waiting）。

**声明**

BOOL EKI_IsWaiting(*configName[]:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |

**示例**

```
listening = EKI_IsWaiting("VisionCom")
```

### 8.2.27 EKI_IsActive：检查通道是否处于激活状态

**说明**

该函数用于检查通信通道是否处于激活状态（= 连接状态 #Active）。

**声明**

BOOL EKI_IsActive(*configName[]:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |

**示例**

```
active = EKI_IsActive("VisionCom")
```

### 8.2.28 EKI_IsLoaded：检查通道是否已加载

**说明**

该函数用于检查通信通道是否已加载（= 连接状态 <> #NotLoaded）。

**声明**

BOOL EKI_IsLoaded(*configName[]:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |

**示例**

```
loaded = EKI_IsLoaded("VisionCom")
```

### 8.2.29 EKI_IsInState：检查通道是否处于预期状态

**说明**

该函数用于检查通信通道是否处于预期状态。

**声明**

BOOL EKI_IsInState(*configName[]:IN, expectedState:IN*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *expectedState:IN* | 数据类型：ENUM EkiConnectionStates<br>预期状态 |

**数据类型 EkiConnectionStates**

```
ENUM EkiConnectionStates Closed, Waiting, Active,
NotLoaded, Unknown
```

| 状态 | 说明 |
|---|---|
| #Closed | 连接已关闭，无法与对端进行通信。<br>该状态出现在以下事件之后：<br>• EKI_Load<br>• EKI_Close<br>• 发生错误，例如通信故障 |
| #Waiting | EKI 已为通信做好准备，但仍在等待外部边界条件。此时无法与对端通信。<br>仅适用于 EKI 服务器：在执行 EKI_Open 后，等待对端（即发起连接的客户端）接入。 |
| #Active | 与对端已成功建立连接。此时可以正常发送和接收数据。 |
| #NotLoaded | 请求的通信通道尚未被加载。 |
| #Unknown | 连接状态未知，因为无法进行查询。 |

**示例**

```
inState = EKI_IsInState("VisionCom", #Active)
```

## 8.2.30 EKI_HasValue：检查可选值是否存在

**说明**

该函数用于检查在选定的接收数据包中是否存在被配置为可选的值。

**声明**

INT EKI_HasValue(*configName[]:IN, valueName[]:IN, value:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *valueName[]:IN* | 数据类型：CHAR<br>要检查其存在性的可选值的名称 |
| *value:OUT* | 数据类型：BOOL<br>检测结果<br>• TRUE：值存在于接收数据包中。<br>• FALSE：值不存在。 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_HasValue("VisionCom", "PosX", ValueExists)
```

## 8.2.31 EKI_HasFrameValue：检查是否存在完整的 KRL 坐标系

**说明**

该函数用于检查在选定的接收数据包中是否存在完整的 KRL 坐标系。

当接收数据包中包含所有六个相关的 REAL 分量（X、Y、Z、A、B、C）时，该坐标系即被视为完整。检查时只需指定坐标系名称即可。

**声明**

INT EKI_HasFrameValue(*configName[]:IN, frameName[]:IN, value:OUT*)

**参数**

| 参数 | 说明 |
|---|---|
| *configName[]:IN* | 数据类型：CHAR<br>以太网配置名称 |
| *frameName[]:IN* | 数据类型：CHAR<br>待检查其各分量的坐标系名称 |
| *value:OUT* | 数据类型：BOOL<br>检测结果<br>• TRUE：所有六个坐标系值均存在于接收数据包中。<br>• FALSE：至少缺少一个值。 |

**返回值 (RET)**

| 常量 | 信息 |
|---|---|
| EkiOk | - |
| EkiConfigNotFound | *配置“{0}”未加载。* |
| EkiUnexpectedError | *在执行时出现意外错误。代码 {0}* |

**示例**

```
RET = EKI_HasFrameValue("VisionCom", "Position",
ValuesExist)
```

## 8.2.32 更改信息级别

**说明**

消息级别决定了在执行 KRL 函数时输出哪些类型的消息。消息级别可以通过变量 EkiShowMessageLevel 进行调整。

变量在数据列表 Eki_User.dat 中声明：

```
DECL GLOBAL INT EkiShowMessageLevel = 3
```

数据列表的存储位置：

- 在 iiQWorks.Sim 中：在程序编辑器的 R1\TP\EthernetKRL 目录下

- 在 iiQKA.UI 用户界面上：在导航器的 TP\EthernetKRL 目录下

**说明**

变量 EkiShowMessageLevel 可以取以下值：

| 值 | 显示级别 |
|---|---|
| 0 | 不显示任何消息<br>用户仅能通过函数的返回值获知错误信息。 |
| 1 | 仅确认消息 |
| 2 | 确认消息和状态消息 |
| 3 | 所有消息类型：确认消息、状态消息和提示消息 |

**操作步骤**

消息级别可通过多种方式进行调整：

- 直接在数据列表 Eki_User.dat 中修改变量 EkiShowMessageLevel。
- 在 KRL 程序内部修改变量。
- 通过 iiQKA.UI 操作界面上的变量显示功能修改变量。

## 8.3 打开和关闭以太网连接

**说明**

以太网连接既可以由机器人解释器打开和使用，也可以由提交解释器打开和使用。

EKI 作为服务器

- EKI_Open 打开已配置的本地端口。此后，客户端即可连接并发送数据。

EKI 作为客户端

- EKI_Open 打开通往已配置服务器的通信通道。此后即可发送数据。

**示例**

```
 1 DEF Connection()
 2 ...
 3 RET=EKI_Load("Connection")
 4 RET=EKI_Open("Connection")
 5 ...
 6 Write data, send data or get received data
 7 ...
 8 RET=EKI_Close("Connection")
 9 RET=EKI_Unload("Connection")
10 ...
11 END
```

| 行 | 说明 |
|---|---|
| 3 | EKI_Load 加载以太网配置并生成与对端通信所需的所有资源 |
| 4 | EKI_Open 打开通信通道并开始与对端通信 |
| 6 | KRL 指令，用于将数据写入发送缓冲区、发送数据或访问已接收的数据 |
| 8 | EKI_Close 关闭通信通道并结束与对端通信 |

| 行 | 说明 |
| --- | --- |
| 9 | EKI_Unload 卸载以太网道配置 |

## 8.4 读取收到的数据

### 说明

KRL 接收数据的访问方式与接收配置中的格式（XML/字符串）无关。

- 始终通过相应 KRL 变量的已配置名称以相同且类型安全的方式访问数据。
- 访问数据前必须初始化 KRL 变量（例如通过赋值操作）。

### XML 格式示例

接收配置：

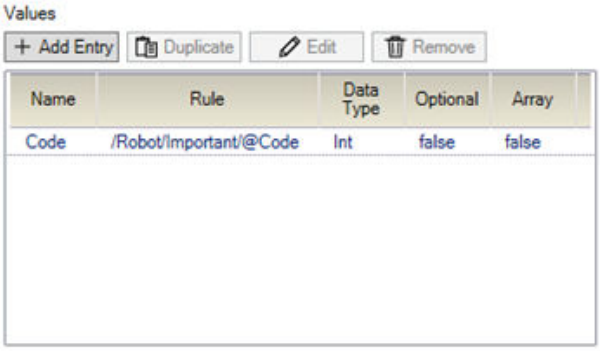



**图 8-1: XML 格式示例**

编程：

```
; Declaration
DECL INT ret
DECL INT recvCode

; Initialization
recvCode = 0

; Access
ret = EKI_ReadNext("Channel_1")
ret = EKI_GetInt("Channel_1", "Code", recvCode)
```

接收到的数据：

```
<Robot>
 <Important Code="110"/>
</Robot>
```

### String 格式示例

**读取固定长度（8 字节）数据包中的文本**

接收配置：

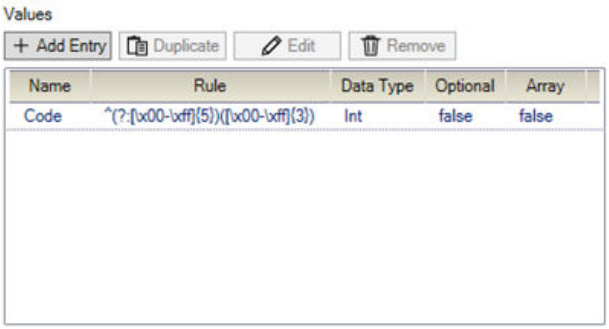



**图 8-2: String 格式示例**

编程:

```
; Declaration
DECL INT ret
DECL INT recvCode

; Initialization
recvCode = 0

; Access
ret = EKI_ReadNext("Channel_1")
ret = EKI_GetInt("Channel_1", "Code", recvCode)
```

接收数据（3 条消息）:

```
Code=110Code=111Code=112
```

**读取在带结束符 (;) 数据包中接收的文本**

接收配置:

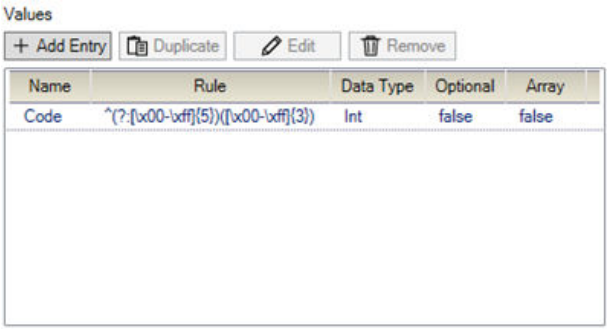



**图 8-3: String 格式示例**

编程:

```
; Declaration
DECL INT ret
DECL INT recvCode

; Initialization
recvCode = 0

; Access
ret = EKI_ReadNext("Channel_1")
ret = EKI_GetInt("Channel_1", "Code", recvCode)
```

接收数据（3 条消息）:

```
Code=110;Code=111;Code=112;
```

### 数组数据示例

(>>> *8.5 "读取数组数据" 页面 60*)

## 8.5 读取数组数据

### 说明

如果接收配置中的 **数组** 参数已激活，则所有接收到的值都将被提取并保存。在 KRL 中只能通过 EKI_Get…Array 函数访问这些数值。
以下示例展示了如何在 KRL 程序中访问数组数据。

### XML 数组示例

接收配置：

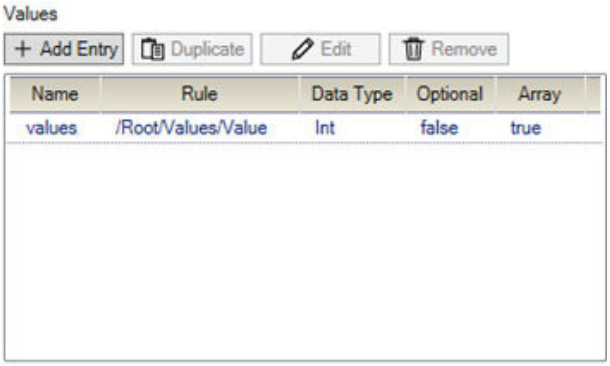



**图 8-4: XML 数组示例**

接收到的数据：

```
<Root>
 <Values>
  <Value>1</Value>
  <Value>2</Value>
  <Value>3</Value>
  <Value>4</Value>
  <Value>5</Value>
  <Value>6</Value>
  <Value>7</Value>
 </Values>
</Root>
```

编程示例 1：处理所有值

```
; Declaration
DECL INT ret, i
DECL INT values[5]
DECL EkiIterator_T iterator

; ...

; Access
ret = EKI_ReadNext("Channel_1")

repeat
 ; loop through all received values
 ; iterator will be increased by the Eki_Get... function
```

```
 ret = EKI_GetIntArray("Channel_1", "values", values[],
iterator)
 for i = 1 to iterator.Count
  ; process received values
  ProceedValue(values[i])
 endfor
until (iterator.Remaining == 0)
```

编程示例 2：处理第 3 至第 7 个值

```
; Declaration
DECL INT ret, i
DECL INT values[5]
DECL EkiIterator_T iterator

; ...

; Access
ret = EKI_ReadNext("Channel_1")

;reads the values from 3-7
iterator.Offset = 2
ret = EKI_GetIntArray("Channel_1", "values", values[],
iterator)
for i = 1 to iterator.Count
 ; process received values
 ProceedValue(values[i])
endfor
```

## 8.6 发送数据

### 说明

发送数据时不区分协议。消息中的所有字符将按照字节逐一发送。

### XML 格式示例

发送配置：

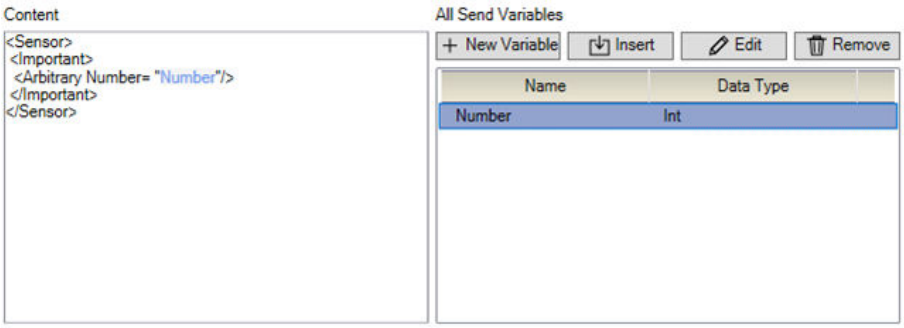



**图 8-5: XML 格式示例**

编程：

```
DECL INT ret
ret = EKI_SetInt("Channel1","Number",13)
ret = EKI_Send("Channel1")
```

发送的数据：

```
<Sensor>
 <Important>
```

```
    <Arbitrary Number="13"/>
  </Important>
</Sensor>
```

### String 格式示例

发送配置：

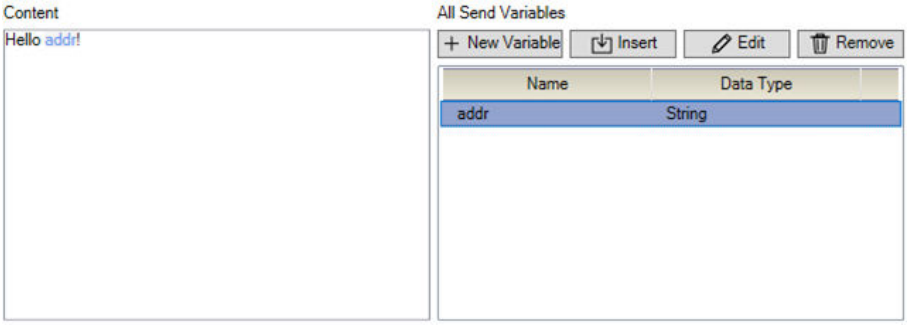



**图 8-6: String 格式示例**

编程：

```
DECL INT ret
ret = EKI_SetString("Channel1","addr","world")
ret = EKI_Send("Channel1")
```

发送的数据：

```
Hello world!
```

# 9 仿真

## 9.1 使用 RCS 进行仿真

**说明**

备选软件包 iiQKA.EthernetKRL 支持在具有基础授权的 iiQWorks.Sim 中使用 RCS 进行仿真时执行 KRL 程序。

使用 RCS 进行仿真时，需注意以下几点：

- 系统不会检查是否存在有效且正确的 EKI 配置文件。
- 系统不会检查 EKI 函数的调用顺序是否正确。

RCS 仿真的默认设置 (*>>> 图 9-1*) 如下：

- 所有 EKI 函数的返回值：返回值 = 0
  返回值“0”表示“EkiOk”（函数已正确执行）。
  此处可查阅所有返回值的列表： (*>>> 8.2 "KRL 函数" 页面 33*)
- EKI 函数的 KRL 值：
  - EKI_GetBool = false
  - EKI_GetInt = 0
  - EKI_GetReal = 0
  - EKI_GetString = ""
  - EKI_HasValue = false
  - EKI_ReadNext = 0
  - EKI_GetConnectionState = Closed

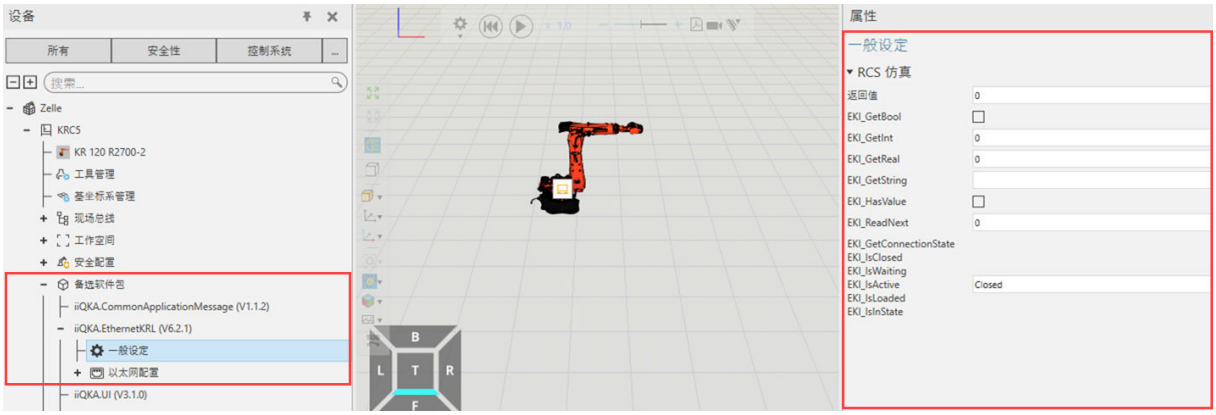



**图 9-1: RCS 仿真通用设置**

仿真所用数值可按如下方式进行设置：

- 在通用设置中输入：适用于整个程序运行的静态数值
- 借助 KRL 程序中的 EKI 注释，可在程序运行期间设定数值。例如，所有 EKI 函数调用的返回值：

```
;@EKI EKI_ReturnValue=0
ret = EKI_Load(configFile[])
if (ret <> EKIOk) THEN
 HALT
ENDIF
```

在 EKI 注释中，可通过下列名称访问相应的设置项：

- 返回值 = EKI_ReturnValue
- EKI_GetBool = EKI_GetBool_OutValue

- EKI_GetInt = EKI_GetInt_OutValue
- EKI_GetReal = EKI_GetReal_OutValue
- EKI_GetString = EKI_GetString_OutValue
- EKI_HasValue = EKI_HasValue_OutValue
- EKI_ReadNext = EKI_ReadNext_OutMsgId
- EKI_GetConnectionState = EKI_GetConnectionState_OutValue

## 示例

程序示例“EKI_Simulation.src”展示了如何在程序运行期间调整仿真所用的数值。该程序并非真实的 EKI 实际应用程序。

(>>> *10 "示例" 页面 67*)

```
 1 DEF  Eki_Simulation ( )
 2  int ret
 3  bool outBool
 4  int outInt
 5  real outReal
 6  char outString[20]
 7  char configFile[20]
 8  int msgId
    ...
19  INI
    ...
31  PTP {A1 0, A2 -90, A3 90, A4 0, A5 0, A6 0}
32
33  outBool = FALSE
34  outInt = 0
35  outReal = 0
36  outString[] = " "
37  configFile[] = "dummy"
38  msgId = -1
39
40  ;@EKI EKI_ReturnValue=0
41  ret = EKI_Load(configFile[])
42  if (ret <> EKIOk) THEN
43   HALT
44  ENDIF
45
46  ret = EKI_Open(configFile[])
47  if (ret <> EKIOk) THEN
48   HALT
49  ENDIF
50
51  ; set connection state to Active
52  ; the connection state in Sim is independent
53  ; of any other EKI calls e.g. Eki_Open/Eki_Close
54
55  ;@EKI EKI_GetConnectionState_OutValue=Active
56  IF (not Eki_IsActive()) THEN
57   HALT
58  ENDIF
59
60  ; Set EKI return value to EkiInvalidOperation
61  ;@EKI EKI_ReturnValue=-6
62  ret = EKI_Send(configFile[])
63  if (ret <> EKIOk) THEN
64   ;@PRINT Simulated error in EKI_Send
65  ENDIF
```

```
66
67  ; set return value to EKIOk
68  ;@EKI EKI_ReturnValue=0
69  ; set id of received message
70  ;@EKI EKI_ReadNext_OutMsgId=2
71  REPEAT
72   ret = EKI_ReadNext(configFile[], msgId)
73  UNTIL ret == EKIOk
74
75  IF (msgId == 2) THEN
76   ;@PRINT Received msg id 2
77  ENDIF
78
79  ;@EKI EKI_HasValue_OutValue=true
80  ret = EKI_HasValue(configFile[], "value", outBool)
81
82  ;@EKI EKI_GetBool_OutValue=true
83  ret = EKI_GetBool(configFile[], "buffer", outBool)
84  ;@EKI EKI_GetInt_OutValue=-50
85  ret = EKI_GetInt(configFile[], "buffer", outInt)
86  ;@EKI EKI_GetReal_OutValue=50
87  ret = EKI_GetReal(configFile[], "buffer", outReal)
88  ;@EKI EKI_GetString_OutValue=any string
89  ret = EKI_GetString(configFile[], "buffer", outString[])
90
91  PTP {A2 outInt, A3 outReal}
92
93  ret = EKI_UnLoad(configFile[])
94  ret = EKI_Close(configFile[])
95
96  PTP {A1 0, A2 -90, A3 90, A4 0, A5 0, A6 0}
97
98 END
```

仿真

# 10 示例

## 10.1 示例概述

### 说明

该备选软件包提供了服务器程序以及各种配置示例和程序示例。

### 前提条件

- 3D 视图中有机器人
- 备选软件包 iiQKA.EthernetKRL 已分配给机器人控制器
- **开始**或**配置**功能区

### 操作步骤

1. 在**设备**窗口中展开机器人控制器的节点。
2. 依次选择节点：**备选软件包** > **iiQKA.EthernetKRL**。
3. 在**属性**窗口中选择**示例**按钮。显示包含示例的目录。
4. 将这些目录复制到本地计算机上。之后，即可从本地计算机将其中包含的配置和程序导入到 iiQWorks.Sim 中。

### 服务器程序

服务器程序可以在外部系统上运行，以建立与机器人控制器的连接。配置示例和程序示例用于测试服务器程序与机器人控制器之间的通信和数据交换。

| 示例 | 目录 |
|---|---|
| 服务器程序<br>• EthernetKRL_Server.exe | …\Application |
| 配置示例<br>• Binary.xml<br>• String.xml<br>• XML.xml | …\Config |
| 程序示例<br>• Binary.src<br>• String.src<br>• XML.src | …\Program |

(>>> *10.1.1 "准备服务器程序和示例" 页面 68*)

### 事件

示例可传输至虚拟/真实的机器人控制器中并在该环境下运行。在 iiQKA.UI 用户界面上会显示程序示例输出的信息提示。

| 示例 | 目录 |
|---|---|
| 配置示例<br>• EventClient.xml<br>• EventServer.xml | …\Config |
| 程序示例<br>• Events.src | …\Program |

### 仿真

该程序示例展示了如何在 KRL 程序中设置用于使用 RCS 进行仿真的数值。

| 示例 | 目录 |
|---|---|
| 程序示例<br>• EKI_Simulation.src | …\Program |

(>>> *9.1 "使用 RCS 进行仿真" 页面 63*)

## 10.1.1 准备服务器程序和示例

### 说明

使用服务器程序和示例需要执行以下准备步骤。

### 前提条件

在 iiQWorks.Sim 中：

- **开始**或**配置**功能区
- 3D 视图中有机器人
- 备选软件包 iiQKA.EthernetKRL 已分配给该配置

在外部系统上：

- 已安装 .NET-Framework 3.5 或更高版本的 Windows 操作系统

### 操作步骤

1. 显示软件 iiQWorks.Sim 中的示例：
   a. 在**设备**窗口中展开机器人控制器的节点。
   b. 选择**备选软件包** > **iiQKA.EthernetKRL**。
   c. 在**属性**窗口中选择**示例**按钮。
2. 将…\Config 和…\Program 下的所有文件复制到本地计算机。
3. 将 …\Application 下的服务器程序复制到外部系统。
4. 将已复制到本地计算机的配置示例（XML 文件）导入到软件 iiQWorks.Sim 中：
   a. 在**设备**窗口中展开机器人控制器的节点。
   b. 选择**备选软件包** > **iiQKA.EthernetKRL** > **以太网配置**。
   c. 点击 **导入配置** 图标，以导入文件。可以同时选择并导入多个文件。
5. 将已复制到本地计算机的程序示例（SRC 文件）导入到软件 iiQWorks.Sim 中：
   a. 切换到**程序**功能区。
   b. 在**程序编辑器**窗口中，用右键点击**程序**文件夹，然后在上下文菜单中选择**导入** > **导入 KRL 文件**。可以同时选择并导入多个文件。
6. 在所需的配置示例中指定运行服务器程序的外部系统的 IP 地址。
   (>>> *6.2 "更改以太网配置的 IP 地址或端口" 页面 22*)
7. 将配置传输到机器人控制器。
8. 启动外部系统上的服务器程序。
9. 点击服务器程序中的菜单按钮，并设置所需的通信属性。
   (>>> *10.1.2 "服务器程序中的操作和设置" 页面 69*)
10. 点击服务器程序中的启动按钮。将建立与机器人控制器的连接。

## 10.1.2 服务器程序中的操作和设置

### 说明

这里描述了用于测试服务器程序和机器人控制器之间通信的操作元件和设置选项。

服务器程序提供以下功能：

- 从机器人控制器接收数据（自动或手动）
- 将数据发送至机器人控制器（自动或手动）
- 显示接收的数据
- 显示发送的数据

### 服务器程序

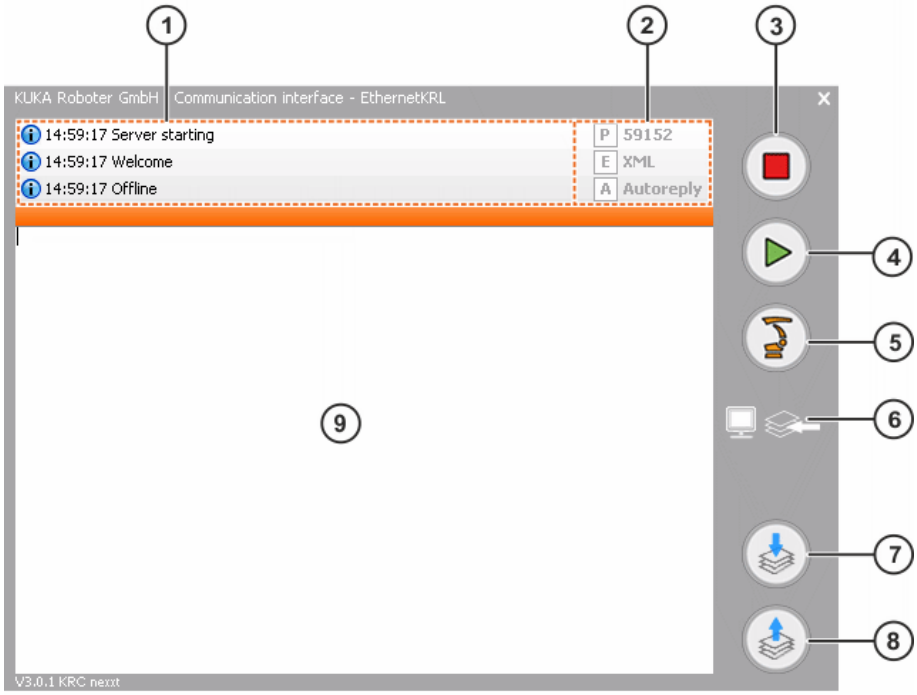



图 10-1：服务器程序

| 序号 | 说明 |
|---|---|
| 1 | 信息窗口 |
| 2 | 显示所设置的通信属性<br>• P：端口编号<br>• E：示例<br>• A：通讯模式<br>如要更改设置，请选择菜单按钮（项号 5）。 |
| 3 | 停止按钮<br>结束与机器人控制器的通讯并且重置服务器。 |
| 4 | 开始按钮<br>分析服务器程序和机器人控制器之间的数据交换。连接第一个到达的连接请求并用作通讯适配器。 |
| 5 | 菜单按钮<br>打开用于设置通信属性的窗口<br>(>>> *"通信属性" 页面 70*) |

| 序号 | 说明 |
|---|---|
| 6 | 显示选项<br>• 箭头向左：显示接收的数据。（默认）<br>• 箭头向右：显示发送的数据。 |
| 7 | 手动接收数据的按钮 |
| 8 | 手动发送数据的按钮 |
| 9 | 显示窗口<br>根据所设置显示选项的不同，显示发送或接收的数据。 |

### 通信属性

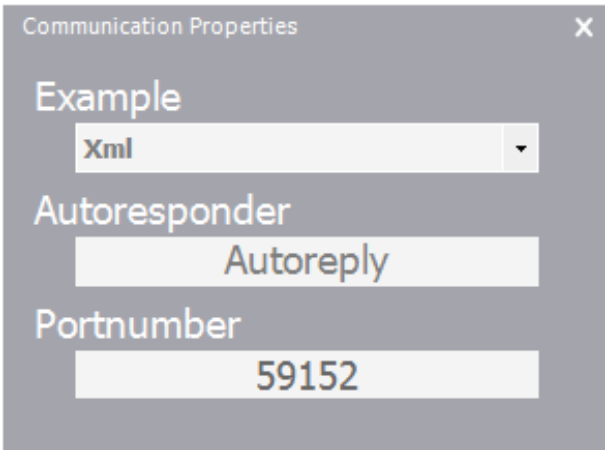



**图 10-2: 通信属性**

| 元素 | 说明 |
|---|---|
| Example | 选择示例。<br>• Xml<br>服务器程序发送和接收 XML 数据。<br>(>>> *10.1.3 "“XML”示例" 页面 71*)<br>• Binary<br>服务器程序发送具有固定长度的二进制数据组。<br>(>>> *10.1.4 "“Binary”示例" 页面 73*)<br>• String<br>服务器程序发送具有可变长度和可变末尾字符串的二进制数据组。<br>(>>> *10.1.5 "“String”示例" 页面 74*)<br>默认：Xml |
| Autoresponder | 选择通信模式。<br>• Autoreply：服务器自动回答每个已接收的数据包。<br>• Manual：仅手动接收数据或发送数据<br>默认：Autoreply |

| 元素 | 说明 |
| --- | --- |
| Portnumber | 输入插座连接的端口编号。<br>默认：59152<br>**注意：**外部系统在该端口上等待机器人控制器的连接请求。必须选择一个空闲号。该号码不得已被其他服务（例如操作系统）使用。 |

### 10.1.3 “XML”示例

**说明**

KRL 程序向服务器发送 XML 数据。随后，KRL 程序从服务器接收 XML 数据并从内存中读取。

发送和接收的数据可在服务器程序窗口中显示。服务器程序中需设置并启动“XML”示例。EKI 配置为客户端。

服务器程序发送以下数据：

```
<Sensor>
 <Message>Example message</Message>
 <Positions>
  <Current X="4645.2" />
  <Before>
   <X>0.9842</X>
  </Before>
 </Positions>
 <Nmb>8</Nmb>
 <Status>
  <IsActive>1</IsActive>
 </Status>
 <Read>
  <xyzabc X="210.3" Y="825.3" Z="234.3" A="84.2" B="12.3" C="43.5" />
 </Read>
 <Show error="0" temp="9929">Taginfo in attributes</Show>
 <Free>2912</Free>
</Sensor>
```

图 10-3: XML 数据

**KRL 程序**

```
 1 DEF XML( )
 2 DECLARATION
   ...
15 INI
   ...
23 INITIALIZE SAMPLE DATA
   ...
34 RET=EKI_Load("XML")
35 RET=EKI_Open("XML")
36
37 ;FOLD Write data to connection
38   RET=EKI_SetReal("XML","X", sendFrame.X)
39   RET=EKI_SetReal("XML","Y", sendFrame.Y)
40   RET=EKI_SetReal("XML","Z", sendFrame.Z)
41   RET=EKI_SetReal("XML","A", sendFrame.A)
42   RET=EKI_SetReal("XML","B", sendFrame.B)
43   RET=EKI_SetReal("XML","C", sendFrame.C)
44   RET=EKI_SetReal("XML","ActX", 1000.12)
45   RET=EKI_SetInt("XML","Status", 12345678)
```

示例

```
46    RET=EKI_SetString("XML","Mode","ConnectSensor")
47    RET=EKI_SetBool("XML","LightOn",true)
48 ;ENDFOLD (Write data to connection)
49 ;FOLD Send data to external program
50    RET = EKI_Send("XML")
51 ;ENDFOLD (Send data to external program)
52
53 REPEAT
54    RET = EKI_ReadNext("XML")
55 UNTIL RET == EkiOk
56
57 ;FOLD Get received sensor data
58    RET=EKI_GetString("XML","Message",valueChar[])
59    RET=EKI_GetReal("XML","CurrentX",valueReal)
60    RET=EKI_GetReal("XML","BeforeX",valueReal)
61    RET=EKI_GetInt("XML","Number",valueInt)
62    CAM_tMsg = {Txt[] "Example received data: Number: %1",Modul[] "EKI
   XML",Nr 1,Param1{I valueInt}}
63    CAM_SendMsg(CAM_tMsg)
64    RET=EKI_GetBool("XML","IsActive",valueBOOL)
65    RET=EKI_GetReal("XML","Frame/X",valueFrame.X)
66    RET=EKI_GetReal("XML","Frame/Y",valueFrame.Y)
67    RET=EKI_GetReal("XML","Frame/Z",valueFrame.Z)
68    RET=EKI_GetReal("XML","Frame/A",valueFrame.A)
69    RET=EKI_GetReal("XML","Frame/B",valueFrame.B)
70    RET=EKI_GetReal("XML","Frame/C",valueFrame.C)
71    RET=EKI_GetBool("XML","ErrorOccurred",valueBOOL)
72    RET=EKI_GetInt("XML","temp",valueInt)
73    RET=EKI_GetString("XML","Show",valueChar[])
74    RET=EKI_GetInt("XML","Free",valueInt)
75    CAM_tMsg = {Txt[] "End get received sensor data",Modul[] "EKI XML",Nr 2}
76    CAM_SendMsg(CAM_tMsg)
77 ;ENDFOLD (Get received sensor data)
78
79 RET=EKI_Close("XML")
80 RET=EKI_Unload("XML")
81 END
```

| 行 | 说明 |
|---|---|
| 2 | 使用的 KRL 变量的声明（折叠） |
| 23 | 通过赋值将 KRL 变量初始化（折叠） |
| 34 | EKI_Load 加载以太网配置 |
| 35 | EKI_Open 打开通信通道并开始与对端通信 |
| 37 … 48 | 使用 EKI_Set 函数将 XML 数据按类型写入发送缓冲区 |
| 49 … 51 | EKI_Send 将 XML 数据发送至服务器 |
| 53 … 55 | EKI_ReadNext 选择服务器接收的下一个数据包 |
| 57 … 77 | 在折叠中通过 EKI_Get 函数从选定数据包中按数据类型读取 XML 数据 |
| 79 | EKI_Close 关闭通信通道并结束与对端通信 |
| 80 | EKI_Unload 卸载以太网道配置 |

## 10.1.4 “Binary”示例

### 说明

KRL 程序向服务器发送固定长度的二进制数据。随后，KRL 程序从服务器接收固定长度的二进制数据。根据配置，二进制数据组具有固定长度 10。

在 KRL 程序中定义了以下数据类型：

- 实数值：34.425
- 整数值：674345
- 字符：R
- 布尔值：True

这些数据类型在 KRL 程序中被转换为二进制数据组，随后发送至服务器程序。

发送和接收的数据可在服务器程序窗口中显示。服务器程序中需设置并启动“Binary”示例。EKI 配置为客户端。

通过“ReceivedData”访问 KRL 程序从服务器接收的数据。这些数据随后同样需要被反向转换。

即使通过 EKI 传输的是二进制数据，EKI 在内部也会将其作为字符串进行处理。因此，在以太网配置中必须将二进制数据声明为字符串类型。尽管如此，在 KRL 程序中仍需将这些数据重新反向转换回其原始的二进制格式。

### KRL 程序

```
 1 DEF Binary( )
 2 DECLARATION
14 INI
23 INITIALIZE SAMPLE DATA
   ...
34 RET=EKI_Load("Binary")
35 RET=EKI_Open("Binary")
36
37 OFFSET=0
38 CAST_TO(Bytes[],OFFSET,34.425,674345,"R",TRUE)
39 SendSize = 4 + 4 + 1 + 1 ;real + int + char + bool
40 RET = EKI_SetString("Binary", "SendData", Bytes[], SendSize)
41 RET = EKI_Send("Binary")
42
43 REPEAT
44  RET = EKI_ReadNext["Binary")
45 UNTIL RET == EkiOk
46
47 RET=EKI_GetString("Binary","ReceivedData",Bytes[])
48
49 OFFSET=0
50 CAST_FROM(Bytes[],OFFSET,valueReal,valueInt,valueChar[],valueBool)
51 CAM_tMsg = {Txt[] "Example received data: valueInt: %1",Modul[] "EKI
   Binary",Nr 1,Param1{I valueInt}}
52 CAM_SendMsg(CAM_tMsg)
53
54 RET=EKI_Close("Binary")
55 RET=EKI_Unload("Binary")
56 END
```

| 行 | 说明 |
|---|---|
| 2 | 使用的 KRL 变量的声明（折叠） |
| 23 | 通过赋值将 KRL 变量初始化（折叠） |

| 行 | 说明 |
|---|---|
| 34 | EKI_Load 加载以太网配置 |
| 35 | EKI_Open 打开通信通道并开始与对端通信 |
| 38 | CAST_TO 将数据转换为二进制数据并写入 CHAR 字段 Bytes[] 中 |
| 39 | SendSize 计算 CHAR 字段 Bytes[] 的大小 |
| 40 | EKI_SetString 将 CHAR 字段作为字符串设置到发送缓冲区 |
| 41 | EKI_Send 将字符串发送到服务器 |
| 43 … 45 | EKI_ReadNext 选择服务器接收的下一个数据包 |
| 47 | EKI_GetString 从选定的数据包中读取接收值 |
| 50 | CAST_FROM 读取 CHAR 字段 Bytes[] 中的值，并按类型复制到指定变量 |
| 51 … 52 | 在 iiQKA.UI 操作界面上输出“valueInt”的接收值 |
| 54 | EKI_Close 关闭通信通道并结束与对端通信 |
| 55 | EKI_Unload 卸载以太网道配置 |

### 10.1.5 “String”示例

**说明**

KRL 程序向服务器发送可变长度的字符串数据。随后，KRL 程序从服务器接收可变长度的字符串数据。在此示例中，发送的字符串以 CR、LF 标记结尾。

发送和接收的数据可在服务器程序窗口中显示。服务器程序中需设置并启动“String”示例。EKI 配置为客户端。

**KRL 程序**

```
 1 DEF String( )
 2 DECLARATION
   ...
 8 INI
   ...
18 INITIALIZE SAMPLE DATA
   ...
24 RET=EKI_Load("String")
25 RET=EKI_Open("String")
26
27 Bytes[]="Stream ends with CR,LF"
28
29 RET=EKI_SetString("String", "SendData", Bytes[])
30 RET=EKI_Send("String")
31
32 REPEAT
33  RET=EKI_ReadNext("String")
34 UNTIL RET==EkiOk
35
36 RET=EKI_GetString("String","ReceivedData",Bytes[])
37 RET=EKI_Close("String")
38 RET=EKI_Unload("String")
39 END
```

| 行 | 说明 |
|---|---|
| 2 | 使用的 KRL 变量的声明（折叠） |
| 18 | 通过赋值将 KRL 变量初始化（折叠） |

| 行 | 说明 |
| --- | --- |
| 24 | EKI_Load 加载以太网配置 |
| 25 | EKI_Open 打开通信通道并开始与对端通信 |
| 27 | 用数据描述 CHAR 字段 Bytes[] |
| 29 | EKI_SetString 将 CHAR 字段作为字符串设置到发送缓冲区 |
| 30 | EKI_Send 将字符串发送到服务器 |
| 32 … 34 | EKI_ReadNext 选择服务器接收的下一个数据包 |
| 36 | EKI_GetString 从选定的数据包中读取接收值 |
| 37 | EKI_Close 关闭通信通道并结束与对端通信 |
| 38 | EKI_Unload 卸载以太网道配置 |

示例

# 11 诊断

## 11.1 使用诊断监视器进行通道诊断

**说明**

在 iiQKA.UI 用户界面的诊断监视器中，为每个通信通道提供了一个独立的模块。在该模块中会显示以下诊断数据：

| 名称 | 说明 |
|---|---|
| **信道名称** | 通信通道名称 |
| **连接类型** | 连接类型<br>• Client：EKI 接口作为客户端<br>• Server：EKI 接口作为服务器 |
| **状态** | 通信通道的连接状态 |
| **对端 IP 地址** | 对端（客户端或服务器）的 IP 地址 |
| **对端的端口** | 对端（客户端或服务器）的端口 |
| **接收到的消息** | 已接收消息的数量 |
| **未解析的字节数** | 未处理的字节数 |

**操作步骤**

1. 在主菜单中选择**诊断 & 监控 > 诊断显示器**。
2. 在**模块**栏位中选择一个通信通道。显示相应的诊断数据。
3. 运行所需程序。可变诊断数据的值在发生变化时会自动更新。

# 12 附录

## 12.1 迁移至 iiQKA.EthernetKRL 6.x 的编程说明

自 6.0 版本起，备选软件包 iiQKA.EthernetKRL 整体上比该软件选项的早期版本更易于理解且结构更清晰。如今，EKI 相关的 KRL 编程方式不再受发送或接收的数据格式的影响，始终保持统一。

### 12.1.1 通用变更

**编程**

- 全球结构变量 EKI_Status 将由一个 INT 错误代码所替代，该代码作为相应 EKI 函数的返回值。
- 数据缓冲机制已进行根本性重构。接收到的数据不再单独且彼此独立地存储。取而代之的是，一条完整接收到的消息中的所有数据将被统一存放。

  这样一来，对消息数据的访问能够始终保持一致。仅当消息完整（即包含所有已配置的元素）时，才会对其进行存储。
- 现在，在 KRL 中等待新接收的数据包并进行查询的流程如下：

```
REPEAT
 RET = EKI_ReadNext("myEKIConfig")
UNTIL RET == EkiOk
```

  EKI_ReadNext 指令将待读取的下一组数值提供给 KRL，随后将其从接收缓冲区中删除。

  之后，可通过 EKI_Get 函数读取所接收消息中的所有值。

**以太网配置**

- 以太网配置现在可以通过 iiQWorks.App Builder 软件中的图形化编辑器来完成。用户不再需要、同时也无法再手动编辑相关的 XML 配置文件。
- 原先用于指示与对端是否存在连接的“Alive Event”已被取消。取而代之的是“通信错误事件”。
  - 为了报告通信错误，可以定义一个数字输出 ($OUT) 或一个标志 ($FLAG)。
  - 当通过 EKI_Open 打开通信通道时，配置的输出或标志会自动设置为 FALSE。
  - 当“通信错误事件”的值为 TRUE 时，将无法再发送或接收数据。
  - 与原先的“Alive-Event”不同，这一新行为与通信方式（服务器/客户端，TCP/UDP）无关。

### 12.1.2 KRL 函数变更

部分 KRL 函数已被删除，同时新增了 KRL 函数且/或对函数名称进行了更改。

已删除的函数：

- EKI_Check

  不再需要，因为返回值是简单的 INT 类型值，且相关信息直接在函数中输出。
- EKI_GetBoolArray
- EKI_GetIntArray

- EKI_GetRealArray
- EKI_GetFrame
- EKI_GetFrameArray
- EKI_Lock
  不再需要，因为消息现在始终保持一致。
- EKI_Unlock
  不再需要，因为消息现在始终保持一致。

| 旧功能 | 新功能 |
|---|---|
| EKI_Init<br>初始化一个通道，用于进行以太网通讯 | EKI_Load<br>加载 EKI 通道配置 |
| EKI_Clear<br>删除一个通道及所有相关数据存储器，退出以太网连接 | EKI_Unload<br>卸载 EKI 通道配置 |
| EKI_ClearBuffer<br>清除已接收但尚未被读取的数据 | EKI_DeleteReceivedData<br>删除通道中所有已接收的用户数据 |
| EKI_CheckBuffer<br>检查存储器中还有多少数据 | EKI_GetBufferedMsgCount<br>读取接收缓冲区中的消息数量<br>通过 EKI_ReadNext 选择读取的消息不再属于缓冲数据的一部分。 |
| | EKI_ReadNext<br>选择下一个接收的数据包，以便通过 EKI_Get 函数读取 |

为便于现有程序的迁移，EKI_Init 和 EKI_Clear 目前仍受支持。但这二者实际上只是对 EKI_Load 与 EKI_Unload 的调用转发。
原则上不建议继续使用这两个函数。

# 13 KUKA 客户服务部

## Contact

本文档提供了有关操作和控制的信息，并能在故障排除过程中为您提供支持。如需进一步支持，请联系您所在地区的子公司。

各地区子公司的联系方式可在以下网址找到：
www.kuka.com/customer-service-contacts

## 请求

提供技术支持时需要以下信息：

- 问题描述，包括故障持续时间和频率的信息
- 有关整个系统硬件和软件的尽可能全面的信息
  以下列表给出了在许多情况下相关的信息：
  - 运动系统（如机械臂）的型号和序列号
  - 控制器的型号和序列号
  - 管线包的型号和序列号
  - 系统软件的名称和版本
  - 其它软件组件或修改版的名称和版本
  - 系统软件诊断包
    此外，对于 KUKA Sunrise：包括应用在内的现有项目
    对于版本早于 V8 的 KUKA System Software：软件存档（诊断包在此尚不可用。）
  - 所使用的应用程序
  - 所用的附加轴

# 索引

概述，示例 ..... 67
网络安全 ..... 17

B
BOOL 数组，读取 ..... 39

E
EKI ..... 8
EKI_Close ..... 37
EKI_DeleteReceivedData ..... 38
EKI_GetBool ..... 39
EKI_GetBoolArray ..... 39
EKI_GetBufferedMsgCount ..... 51
EKI_GetConnectionState ..... 52
EKI_GetFrame ..... 45
EKI_GetFrameArray ..... 46
EKI_GetInt ..... 40
EKI_GetIntArray ..... 41
EKI_GetReal ..... 42
EKI_GetRealArray ..... 43
EKI_GetString ..... 44
EKI_GetStringArray ..... 44
EKI_HasFrameValue ..... 56
EKI_HasValue ..... 55
EKI_IsActive ..... 53
EKI_IsClosed ..... 53
EKI_IsInState ..... 54
EKI_IsLoaded ..... 54
EKI_IsWaiting ..... 53
EKI_Load ..... 35
EKI_Open ..... 36
EKI_ReadNext ..... 38
EKI_Send ..... 47
EKI_SetBool ..... 48
EKI_SetFrame ..... 51
EKI_SetInt ..... 49
EKI_SetReal ..... 49
EKI_SetString ..... 50
EKI_Unload ..... 37
EthernetKRL_Server.exe ..... 67

F
FIFO ..... 8

I
iiQWorks.App Builder ..... 8
iiQWorks.Sim ..... 8
INT 数组，读取 ..... 41
IP ..... 8

K
KLI ..... 8, 23
KRL ..... 8
KRL 函数 ..... 33
KRL 坐标系，读取 ..... 45
KRL 坐标系，设置 ..... 51
KRL 坐标系数组，读取 ..... 46
KUKA 客户服务部 ..... 81

L
LIFO ..... 8

R
RCS，仿真 ..... 63
REAL 数组，读取 ..... 43

T
TCP ..... 8

U
UDP ..... 8

X
XML ..... 8
XPath ..... 8

安
安全 ..... 15
安全提示 ..... 7
安装 ..... 19
安装，通过 iiQWorks.Sim ..... 19

编
编程 ..... 33

布
布尔值，读取 ..... 39
布尔值，设置 ..... 48

操
操作 ..... 21

产
产品说明 ..... 11

传
传输协议 ..... 12

## 错
错误处理..... 13

## 仿
仿真..... 63

## 服
服务器程序，操作..... 69
服务器程序，设置..... 69
服务器运行模式..... 12

## 附
附录..... 79

## 更
更改信息级别..... 56
更新，通过 iiQWorks.Sim..... 19

## 工
工作原理..... 11

## 功
功能..... 11

## 接
接收到的数据，删除..... 38

## 开
开源..... 9

## 客
客户端运行模式..... 12

## 连
连接状态，查询..... 52

## 目
目标群体..... 7

## 培
培训..... 7

## 配
配置..... 23
配置，概览..... 23

## 实
实数值，读取..... 42
实数值，设置..... 49

## 示
示例..... 67

## 事
事件信息..... 13

## 术
术语，所用的..... 8

## 数
数据，读取..... 58
数据，发送..... 47, 61
数组数据，读取..... 60

## 所
所用术语..... 8

## 套
套接字..... 8

## 提
提示..... 7

## 通
通信通道，打开..... 36
通信通道，关闭..... 37

## 投
投入运行..... 23
投入运行，概览..... 23

## 网
网络连接..... 23

## 违
违规使用..... 13

## 文

文档，工业机器人..... 7

## 系

系统要求..... 19
硬件..... 19

## 下

下一个数据包，选择..... 38

## 消

消息，从接收缓冲区中读取数量..... 51

## 卸

卸载，通过 iiQWorks.Sim.....20

## 许

许可证条件..... 9

## 以

以太网..... 8
以太网连接，打开和关闭..... 57
以太网配置，创建和编辑..... 23
以太网配置，更改 IP 地址或端口.....22
以太网配置，管理..... 21
以太网配置，加载..... 35
以太网配置，卸载..... 37

## 引

引言..... 7

## 预

预期用途..... 13

## 诊

诊断..... 77
诊断包..... 81
诊断监视器，使用..... 77

## 整

整数值，读取..... 40
整数值，设置..... 49

## 知

知识，必需..... 7

## 字

字符串，读取..... 44
字符串，设置..... 50
字符串数组，读取..... 44